

Simulation and Modeling

Concise Study Notes: Chapters 1–10

ENCT 353 Year III, Part II

Scope. These notes follow the full syllabus for Simulation and Modeling (ENCT 353, 45 hours, 60 theory marks): Chapter 1 Introduction to Simulation, Chapter 2 Physical and Mathematical Models, Chapter 3 Simulation of Continuous Systems, Chapter 4 Simulation of Queuing Systems, Chapter 5 Markov Chains, Chapter 6 Random Number Generation (properties, congruential methods, statistical tests, variate generation), Chapter 7 Verification and Validation, Chapter 8 Analysis of Simulation Output, Chapter 9 Simulation Software (Java, GPSS, Python and other tools), and Chapter 10 Simulation of Computer Systems. Every numerical is worked completely, step by step. Chapter 9 includes short illustrative code snippets. Forty MCQs with an answer key are at the end.

Contents

	Chapter	Topics	Page
1	Introduction to Simulation	System concepts; continuous/discrete systems; types of models; model life cycle; steps in simulation study; advantages/disadvantages; Monte Carlo; discrete-event simulation	1
2	Physical and Mathematical Models	Differential/partial differential equations; static and dynamic physical models; static and dynamic mathematical models; mechanical-electrical analogy	9
3	Simulation of Continuous Systems	Continuous system models; analog computer; analog methods; hybrid simulation; digital-analog simulators; CSSLs; feedback systems	13
4	Simulation of Queuing Systems	Elements and characteristics; models and types; Kendall's notation; M/M/1 performance measures; network of queues; applications	19
5	Markov Chains	Markov property; transition matrices; n -step transitions; steady state; applications	27
6	Random Number Generation	Properties; pseudo-random generation; congruential methods; K-S, chi-square, gap, poker and autocorrelation tests; discrete-distribution generation; inversion, rejection, composition, convolution	31
7	Verification and Validation	Verification; calibration and validation; Naylor and Finger process; validation errors	44
8	Analysis of Simulation Output	Confidence intervals and hypothesis testing; estimation methods; run statistics; replication; initial-bias elimination	47
9	Simulation Software	Java, GPSS, Python and other simulation tools, with short code examples	51
10	Simulation of Computer Systems	Simulation tools; high-level computer-system simulation; CPU pipeline; memory (LRU); network simulation	56
	Knowledge check	40 MCQs with answer key at the end	60

Chapter 1 Introduction to Simulation

1.1 System and System Environment Concepts

System

A **system** is a collection of entities (people, machines, components) that act and interact together, in a regular and interdependent way, toward the accomplishment of some logical end (Klir, 1969). It exists and operates in time and space, bounded by a **system boundary**. *Example:* a computer, a communication network, a queueing system.

Three basic implications: (1) a system is designed for a predetermined objective; (2) interrelationship and interdependence exist among its components; (3) the objectives of the whole system outrank those of any individual subsystem.

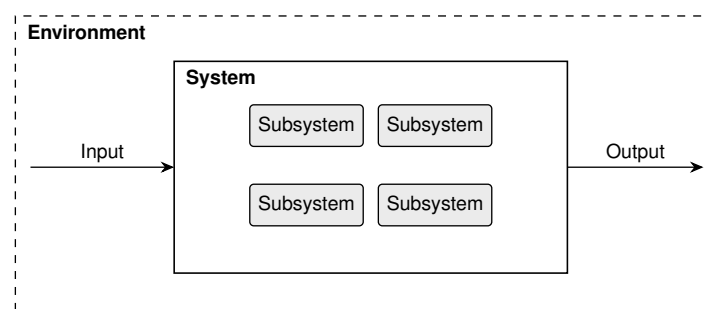


Figure 1: A system, its subsystems, and its environment.

System components

Entity: an object of interest that passes through the system (e.g. a car in a traffic system). **Attribute:** a property or characteristic of an entity. **Activity:** a process that causes a change in the system. **Event:** an instantaneous occurrence that may change the state of the system. **State:** the collection of variables necessary to describe the system at a given time.

System	Entities	Attributes	Activities	Event
Traffic	Cars, buses	Speed, distance	Driving, stopping	Arrival, departure
Bank	Customers	Balance, credit status	Depositing cash, withdrawal	Arrival, departure
Communication	Message	Length, priority	Transmitting message	Message breakdown
Supermarket	Customers	Shopping list	Clearance (checkout)	Arrival, departure

System environment: external factors that influence the system but are not part of it — a system affects, and is affected by, its environment; whatever it cannot control (e.g. what drives customer arrivals at a supermarket) belongs to the environment.

Activities are classified two ways. By place: **endogenous** (within the system, e.g. cheque clearing) vs. **exogenous** (between system and environment, e.g. customers arriving). By predictability: **deterministic** (outcome fully fixed by the input, e.g. switch off → no light) vs. **non-deterministic/stochastic** (outcome varies randomly, described by a probability distribution, e.g. power failure).

1.2 Continuous and Discrete Systems

Continuous vs. discrete systems

A **continuous system** is one whose state variables change *continuously* with time. *Example:* room temperature, aircraft altitude/speed in flight. A **discrete system** is one whose state variables change *instantaneously*, at separated points in time, and stay constant in between. *Example:* a bank queue (length jumps by whole customers), a traffic light.

Aspect	Discrete	Continuous
State change	Changes abruptly at discrete points in time	Changes continuously with time
Viewed as	Changing only at discrete points in time	Changing continuously across time
Associated with	An event at each state transition; events carry a time stamp	Typically described by a set of differential equations

1.3 Types of Models

Model

A **model** is a simplified representation of a system, at some particular point in time or space, meant to promote understanding of the real system and let the analyst predict the effect of changes to it — a trade-off between *realism* and *simplicity*: close enough to allow valid conclusions, not so complex that it becomes unusable.

Modeling has two tasks: **establishing model structure** (fixing the boundary, entities, attributes, activities) and **supplying the data** (values for attributes, relationships for activities).

Classification by nature of the system. A system model is first split into **deterministic** and **stochastic**, then each into **static** and **dynamic**; the dynamic branches split again into **continuous** and **discrete**. This gives the standard names for simulation techniques:

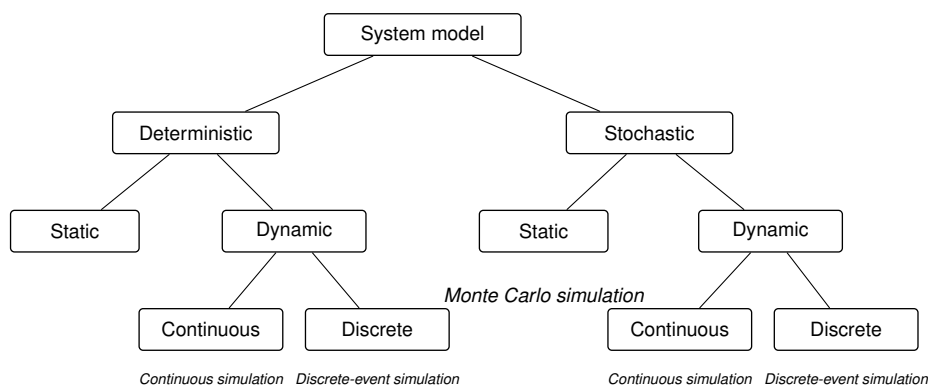


Figure 2: Model classification by nature, with the resulting simulation technique named at each leaf.

Deterministic vs. stochastic, static vs. dynamic

A **deterministic model** has no random elements: a known input always gives the same, unique output. *Example:* a digital circuit. A **stochastic model** has random input variables, so its output is itself a random quantity, needing several runs to analyse. *Example:* service time of customers at a shop. A **static model** represents a system at one instant, or one that does not change with time. *Example:* the odds of a winning solitaire hand. A **dynamic model** represents a system as it evolves over time. *Example:* a traffic simulation.

Classification by representation. Models → **Physical** / **Mathematical** → each splits into Static / Dynamic → the mathematical branches split again into **Numerical** / **Analytical**; **system simulation** sits at mathematical → dynamic → numerical.

Physical, mathematical, numerical, analytical

A **physical (iconic) model** is based on an analogy between systems (e.g. mechanical and electrical); attributes are measurements (voltage, shaft position), activities are the physical laws driving it. *Example:* an architectural scale model. A **mathematical model** uses symbolic notation and equations; attributes are variables, activities are functions. A **numerical model** applies a computational procedure to solve equations too complex for a closed form. An **analytical model** solves a problem in closed form by deductive reasoning; exact solutions exist only for sufficiently simple equations. *Example:* a Boolean equation.

Aspect	Static modeling	Dynamic modeling
Relation to runtime	Model of the system <i>not</i> during runtime	Runtime model of the system
Differential equations	Not used; system at equilibrium/steady state	Used; the model keeps changing with reference to time
Composition	Class and object diagrams; depicts static constituents	Sequences of operations, state changes, activities, interactions, memory
Flexibility	Rigid; time-independent view; cannot change in real time	Flexible; can be changed in real time

1.4 Model Development Life Cycle

Model development life cycle

The step-by-step process used to build a simulation model of a real system, helping the analyst understand the system, build the model, and test how it behaves. It is *fundamentally an iterative process*: later stages routinely send the modeler back to an earlier one.

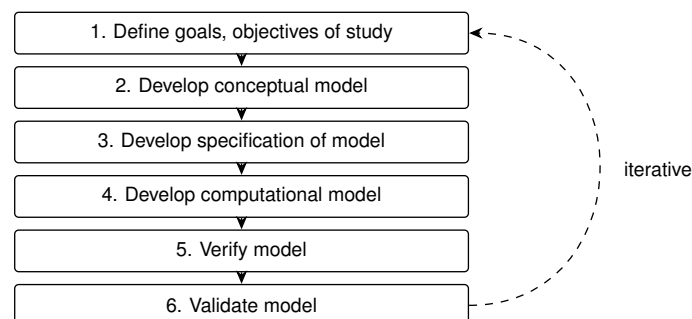


Figure 3: Model development life cycle.

- 1. Define goals, objectives of study** — the objective of the simulation study for the chosen system.
- 2. Develop conceptual model** — study the real system's entities and attributes and build a conceptual model.
- 3. Develop specification of model** — gather detail: attributes, required data, algorithms (e.g. for traffic: road geometry, signal timing, demand, driver behaviour).
- 4. Develop computational model** — build the executable model in a general-purpose or simulation language, or a simulation package.
- 5. Verification** — “Did I build the model right?” Matches the computational model to the specification; largely debugging.
- 6. Validation** — “Did I build the right model?” Matches the computational model to the real system, via measurement or an analytical model; by necessity always incomplete.

Verification = built the model right (internal correctness). **Validation** = built the right model (external correctness). These reappear as the “Verified?”/“Validated?” checks in §1.5's diagram.

1.5 Simulation and Steps in Simulation Study

Simulation

A **simulation** is the operation of a model of a system: a computer experiment in which the real system is replaced by the execution of a program that mimics its behaviour. It manipulates a model so that it operates on time or space, compressing or expanding it, so the analyst can perceive interactions that would otherwise be hidden by their separation in time or space.

When to use it. Before altering a system or building a new one, to catch failures to meet specification, eliminate bottlenecks and optimise performance — especially when the real system is too hazardous, costly or inaccessible to observe directly. **When not to.** If the problem has an analytical solution, if direct experiment is easier, or if the time, resources or data for verification and validation are unavailable.

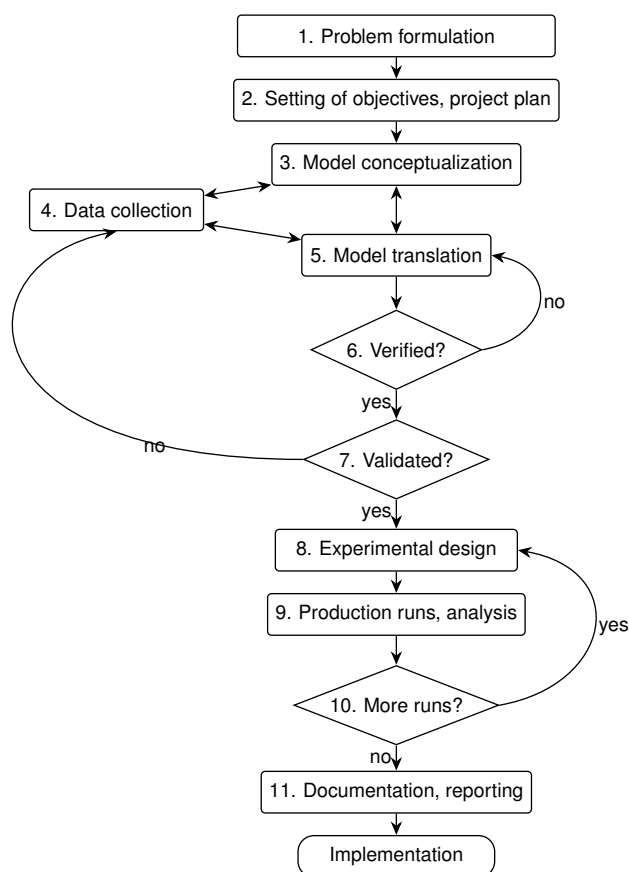


Figure 4: Steps in a simulation study, with the verify/validate and more-runs feedback loops.

1. **Problem formulation** — a clear statement of the problem, the questions to answer, and the measurements needed.
2. **Setting of objectives** — the questions simulation should answer; the project plan lists alternatives, cost and time required.
3. **Model conceptualization** — start simple and add complexity gradually: abstract the essential features and enrich until it is a useful approximation.
4. **Data collection** — begins alongside early model building; the data required changes as the model does.
5. **Model translation** — enter the model in computer-recognisable form, via a simulation language or software.
6. **Verified?** — is the program performing properly? Complete once the input parameters and logic are correctly represented.
7. **Validated?** — usually achieved through calibration against real data.

8. **Experimental design** — decide which alternatives to simulate, based on runs already analysed.
9. **Production runs and analysis** — estimate performance measures for the designs simulated.
10. **More runs?** — decide, from the analysis so far, whether more runs (and what design) are needed.
11. **Documentation and reporting** — program documentation plus a progress report, ending in implementation.

1.6 Advantages and Disadvantages of Simulation

Advantages. Study a system without building it (wind-tunnel testing a model aircraft), including cases too costly or risky on the real system. *Time compression/expansion*: slow processes compress into minutes of computer time; very fast ones (millions of events/second in a reactor) expand for study. Lets management foresee bottlenecks before committing to new equipment. Models are flexible and easy to modify; good software is widely available; an excellent training tool (flight simulators).

Disadvantages. Model building is a skill learned through experience — two modelers rarely build identical models. Outputs are random variables, so results can be hard to interpret. Sometimes used even when an analytical solution would be preferable; modeling/analysis can be time-consuming and expensive. It only *evaluates* solutions, it does not generate them, and being trial-and-error it is imprecise — a set of responses, not one exact answer.

Areas of application (representative): manufacturing, business, military, healthcare, communication networks, computer systems, economics, transportation, environment, and biology (epidemics, population genetics).

1.7 Monte-Carlo Simulation

Monte Carlo simulation

Performing sampling experiments on a model of a system is called **stochastic simulation**; because it samples from a probability distribution using random numbers, it is commonly called **Monte Carlo simulation**. It is a computational technique that uses random sampling to solve problems and estimate numerical results, especially where an analytical solution is difficult or impossible.

Procedure. (1) *Define the problem* — identify the uncertain variables and their probability distributions. (2) *Generate random values* — draw random samples from each distribution using a random-number generator. (3) *Run the simulation* — compute the outcome for each random sample, repeated many times. (4) *Analyse the results* — collect the outcomes and compute statistics (mean, standard deviation, percentiles).

Example: Estimating π . A unit circle (area π) is inscribed in a square of side 2 (area 4), so for a point chosen uniformly in the square, $P(\text{inside circle}) = \pi/4$, giving $\pi \approx 4 \times (\text{points with } x^2 + y^2 \leq 1) / (\text{total points})$. Generate random (x, y) in the square, test $x^2 + y^2 \leq 1$, repeat many times, and take $4 \times$ the hit ratio. As N grows the estimate converges to $\pi \approx 3.14159$: $N=10 \Rightarrow \approx 3.2$; $N=1,000 \Rightarrow \approx 3.142$; $N=1,000,000 \Rightarrow \approx 3.141592$.

Example: Dentist appointment scheduling. Sita schedules every patient for a 30-minute slot starting 8:00 am; the actual time needed depends on the category of work.

Category	Time (min)	Patients	Probability	RN interval
Filling	45	40	0.40	00–39
Crown	60	15	0.15	40–54
Cleaning	15	15	0.15	55–69
Extracting	45	10	0.10	70–79
Checkup	15	20	0.20	80–99

Probability = patients/100 (checks: $40 + 15 + 15 + 10 + 20 = 100$, and $0.40 + 0.15 + 0.15 + 0.10 + 0.20 = 1.00$). Cumulative probability: 0.40, 0.55, 0.70, 0.80, 1.00, which fixes the two-digit random-number (RN) intervals in the table above. Eight patients arrive exactly on schedule, every 30 minutes from 8:00 am, with random numbers 40, 82, 11, 34, 25, 66, 17, 79; each service starts at $\max(\text{scheduled arrival, previous service end})$, since the doctor serves one patient at a time.

Pt.	Sched.	RN	Category (time)	Start	End	Wait (min)
1	8:00	40	Crown (60)	8:00	9:00	0
2	8:30	82	Checkup (15)	9:00	9:15	30
3	9:00	11	Filling (45)	9:15	10:00	15
4	9:30	34	Filling (45)	10:00	10:45	30
5	10:00	25	Filling (45)	10:45	11:30	45
6	10:30	66	Cleaning (15)	11:30	11:45	60
7	11:00	17	Filling (45)	11:45	12:30	45
8	11:30	79	Extracting (45)	12:30	1:15	60

Each Start is $\max(\text{scheduled arrival, previous End})$, and each Wait is $\text{Start} - \text{Scheduled}$. Every service starts exactly when the previous one ends or later, so the doctor is never idle: **idle time** = 0 throughout (patient 1's 60-minute crown already overruns its 30-minute slot, and this particular random sequence keeps the queue backed up, pushing patient 8's service to 1:15 pm — past the nominal 4-hour window). Total waiting time = $0 + 30 + 15 + 30 + 45 + 60 + 45 + 60 = 285$ minutes, so

$$\text{Average waiting time} = \frac{285}{8} = 35.625 \text{ minutes.}$$

The same RN-interval technique applies directly to inventory/demand problems: a demand distribution's cumulative probabilities fix RN intervals exactly as above, random numbers map to a sequence of simulated daily demands, and averaging that sequence estimates mean demand (and, against a fixed production quantity, expected leftover stock).

Practice problem (unsolved, same style): an air-force station schedules 15 flights a day, each needing one pilot, with 3 reserve pilots available; given a probability distribution for pilots falling sick (0 to 5, probabilities 0.15, 0.20, 0.25, 0.15, 0.15, 0.10), simulate 5 days to estimate reserve-pilot utilisation and the probability of cancelling a flight.

1.8 Discrete-Event System Simulation

Discrete-event simulation: clock, event, event list

Discrete-event system simulation models a system whose **state** changes only at a discrete set of points in time, called **events**, rather than continuously. *Example:* customer arrivals in a bank, packet arrivals on a network, traffic-light changes. The **simulation clock** is a variable holding the current simulated time. The **event list** (future event list) stores every scheduled future event with its event time, kept sorted so the next event is always at the head. Under the **next-event time-advance mechanism**, the clock jumps directly to the time of the next event in the list (skipping the idle time in between); that event is processed — updating the state and possibly scheduling new future events — and the cycle repeats until the list is empty or a stopping condition is met.

Example: A tiny single-server queue. Three customers, C_1, C_2, C_3 , arrive (exogenously) at times 2, 5, 10 with service times 4, 3, 2; the server handles one customer at a time, in arrival order.

Clock	Event	Action taken	Server	Event list after
0	—	initialise	idle	$\{A_1=2, A_2=5, A_3=10\}$
2	Arrival C_1	start C_1 ; sched. $D_1=6$	busy (C_1)	$\{A_2=5, A_3=10, D_1=6\}$
5	Arrival C_2	C_2 queues (server busy)	busy (C_1)	$\{A_3=10, D_1=6\}$
6	Departure C_1	C_1 done; start C_2 ; sched. $D_2=9$	busy (C_2)	$\{A_3=10, D_2=9\}$
9	Departure C_2	C_2 done; queue empty	idle	$\{A_3=10\}$
10	Arrival C_3	start C_3 ; sched. $D_3=12$	busy (C_3)	$\{D_3=12\}$
12	Departure C_3	C_3 done; list empty	idle	$\{ \}$ (ends)

The clock advances $0 \rightarrow 2 \rightarrow 5 \rightarrow 6 \rightarrow 9 \rightarrow 10 \rightarrow 12$ — straight from one event to the next, never pausing at the empty instants in between — exactly what separates discrete-event simulation from a continuous, differential-equation-based one that steps through time in small fixed increments.

Chapter 2 Physical and Mathematical Models

2.1 Differential and Partial Differential Equations

Differential equation

A **differential equation** relates an unknown function to its derivatives, essentially describing how a quantity changes with respect to another. It is used extensively in science, engineering and economics to model change over time and space: population growth, spread of disease, motion of vehicles, temperature change, financial interest.

Ordinary vs. partial differential equations

An **ordinary differential equation (ODE)** contains derivatives with respect to a single variable. *Example:* $\frac{dy}{dx} + xy = 3y$. A **partial differential equation (PDE)** contains partial derivatives of a function of two or more independent variables — e.g. for $z = f(x, y)$, differentiating only with respect to x while holding y fixed gives $\partial z / \partial x$. *Example:* $x^2 \frac{\partial z}{\partial x} + xy \frac{\partial z}{\partial y} = a$. PDEs describe quantities that change with space, time, or both: sound, heat, electrostatics, fluid dynamics, elasticity, quantum mechanics.

Example: Population growth. If a population grows in proportion to its current size,

$$\frac{dP}{dt} = kP, \quad P = \text{population}, \quad k = \text{growth constant}.$$

This is a first-order linear ODE. Separating variables, $\frac{dP}{P} = k dt$; integrating both sides, $\ln P = kt + C$; exponentiating, $P = e^{kt+C} = e^C e^{kt}$; and writing $P_0 = e^C$ for the population at $t = 0$,

$$P(t) = P_0 e^{kt},$$

the familiar exponential growth model ($k < 0$ gives decay instead).

2.2 Static Physical Model

Static physical model

Also called an **iconic model** (“look-alike” model): a scaled-down model of a system that does not change with time.

Example: before constructing a building, an architect builds a scaled-down model reflecting all its rooms, outer design and other important features — a static physical model. Other examples: mechanical scale models; scale models used in wind tunnels and water tanks when designing aircraft and ships. **Scaling** — positive (enlarging) or negative (shrinking) — is what turns a real system into a workable static physical model; a house model, for instance, gives a simple way to read off exact height and width measurements once the scale is fixed. Such models can sometimes also be used to solve equations directly, for particular boundary conditions.

2.3 Dynamic Physical Model

Dynamic physical model

Relies on an analogy between the system being studied and some other system of a different nature, the analogy usually resting on an underlying similarity in the forces that govern the behaviour of both systems.

Example: at a pipe junction, two inflows $Q_a = 3$ and $Q_b = 9$ combine into one outflow, $Q_c = Q_a + Q_b = 3 + 9 = 12$ — the same conservation law as Kirchhoff’s current law at a circuit node, $I_3 = I_1 + I_2$: hydraulics and simple circuit laws are analogous, and by extension so is Ohm’s law. The same style of reasoning-by-analogy is used more speculatively too: because the Sun’s mass exceeds Earth’s and attracts it, Earth revolves around the Sun; a nucleus’s mass likewise exceeds an electron’s, so it is *plausible* (by analogy, not proof) that an electron revolves around the nucleus — the classical planetary model of the atom.

Mechanical vs. electrical analogy. The most important dynamic physical model in this course is the correspondence between a mechanical mass–spring–damper system and a series electrical RLC circuit — the same underlying physics, in two different-looking forms.

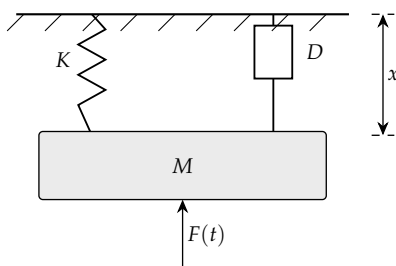


Figure 5: Mechanical mass–spring–damper system: mass M , spring stiffness K , damping factor D , applied force $F(t)$, displacement x .

Mechanical system. Mass M is acted on by a time-varying applied force $F(t)$, a spring of stiffness K , and a damper (dashpot) of damping factor D — for example, a simplified model of a car wheel’s suspension. Newton’s second law gives

$$M\ddot{x} + D\dot{x} + Kx = KF(t),$$

where x is displacement, $\dot{x} = dx/dt$ velocity, and $\ddot{x} = d^2x/dt^2$ acceleration.

Electrical system. A series circuit with inductance L , resistance R and capacitance C , driven by a time-varying voltage source $E(t)$, with q the charge on the capacitor and current $i = \dot{q}$. Kirchhoff’s voltage law gives

$$L\ddot{q} + R\dot{q} + \frac{q}{C} = \frac{E(t)}{C},$$

where $\dot{q} = dq/dt$ is the current and $\ddot{q} = d^2q/dt^2$ its rate of change.

The two governing equations have exactly the same form, term for term, so either system can be studied through the other:

Mechanical quantity	Electrical quantity
Force F	Voltage E
Mass M	Inductance L
Damping factor D	Resistance R
Spring stiffness K	1/Capacitance ($1/C$)
Displacement x	Charge q
Velocity $\dot{x}$	Current $i (= \dot{q})$

Because it is far easier to change a resistor or capacitor than to rebuild a physical suspension, the electrical analog is normally the one actually built: tuning R or C and observing the capacitor voltage oscillate predicts how a different shock absorber or spring would affect the car’s ride, without touching the car itself. (If the mechanical equation were this easy to solve directly, it could equally be treated as a dynamic *mathematical* model instead — see §2.5; the physical analog earns its place once real-world complications make the direct algebra unwieldy.)

2.4 Static Mathematical Model

Static mathematical model

If a mathematical model does not involve time — the system does not change with time — it is a **static mathematical model**. It gives the relationships between system attributes when the system is in equilibrium; if an attribute value is changed, the model gives the *new* equilibrium values for all attributes, but not the path by which the system moved to them. This is the key limitation of a static model next to a dynamic one.

Example: Linear market (supply and demand) model. In marketing a commodity, demand falls and supply rises as price rises:

$$Q = a - bP \text{ (demand),} \quad S = c + dP \text{ (supply),} \quad b, d > 0,$$

where Q is quantity demanded, S quantity supplied, P price, and a, b, c, d are constants fitted from observed data. The market *clears* where supply equals demand, $S = Q$. Solving for the equilibrium price step by step:

$$a - bP = c + dP \implies a - c = bP + dP \implies a - c = (b + d)P \implies P^* = \frac{a - c}{b + d},$$

and the equilibrium quantity is $Q^* = a - bP^*$ (equivalently $S^* = c + dP^*$).

Example: Numerical check. Take $Q = 200 - 4P$ and $S = 20 + 6P$, i.e. $a = 200, b = 4, c = 20, d = 6$. Setting $S = Q$:

$$200 - 4P = 20 + 6P \implies 200 - 20 = 6P + 4P \implies 180 = 10P \implies P^* = 18.$$

Then $Q^* = 200 - 4(18) = 200 - 72 = 128$, and as a check $S^* = 20 + 6(18) = 20 + 108 = 128 = Q^*$. ✓

If demand or supply is a curve rather than a straight line, the two relationships may have no closed-form intersection; the equilibrium is then found numerically — plotting both curves to scale and reading off their point of intersection.

2.5 Dynamic Mathematical Model

Dynamic mathematical model

A **dynamic mathematical model** lets the change of system attributes be derived explicitly as a function of time, by an analytical solution or, when the model is too complex for that, a numerical computation.

Example: Reducing the mass–spring–damper equation to standard second-order form. Starting from §2.3,

$$M\ddot{x} + D\dot{x} + Kx = KF(t).$$

Step 1 — divide every term by M (valid since $M \neq 0$):

$$\ddot{x} + \frac{D}{M}\dot{x} + \frac{K}{M}x = \frac{K}{M}F(t).$$

Step 2 — define the standard second-order parameters, damping ratio ζ and natural angular frequency ω , by $2\zeta\omega = D/M$ and $\omega^2 = K/M$. Substituting turns the equation into the standard second-order form

$$\ddot{x} + 2\zeta\omega\dot{x} + \omega^2x = \omega^2F(t).$$

Step 3 — solve the first definition for ζ :

$$\zeta = \frac{D}{2M\omega}.$$

Example: Condition for non-oscillatory motion. A non-oscillatory response requires $\zeta \geq 1$. Substituting $\zeta = D/(2M\omega)$:

$$\frac{D}{2M\omega} \geq 1 \implies D \geq 2M\omega \quad (\omega > 0) \implies D^2 \geq 4M^2\omega^2.$$

Since $\omega^2 = K/M$, we have $M^2\omega^2 = M^2 \cdot \frac{K}{M} = MK$, so

$$D^2 \geq 4MK.$$

$D^2 > 4MK$: **overdamped** — slow, smooth return to equilibrium, no overshoot. $D^2 = 4MK$: **critically damped** — the fastest return with no overshoot ($\zeta = 1$, the boundary case). $D^2 < 4MK$: **underdamped (oscillatory)** — the response overshoots and oscillates about equilibrium, with the amplitude decaying over time ($\zeta < 1$).

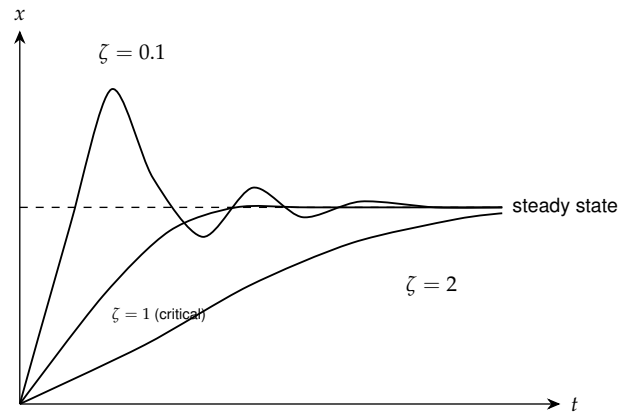


Figure 6: Response $x(t)$ to a suddenly applied steady force, for different damping ratios ζ : underdamped ($\zeta < 1$, oscillates), critically damped ($\zeta = 1$), overdamped ($\zeta > 1$, no oscillation).

For an oscillatory (underdamped) response, the angular frequency of oscillation ω relates to the ordinary frequency f by the standard relation $\omega = 2\pi f$.

Chapter 3 Simulation of Continuous Systems

3.1 Continuous System Models

Continuous system

A **continuous system** is one in which the state variables change continuously with time — equivalently, the predominant activities of the system cause smooth changes in the attributes of its entities (e.g. the flow of water over a dam). When such a system is modelled mathematically the model expresses the *rates* at which attributes change, so it consists of one or more **differential equations**.

If x or any of its derivatives is raised to a power, or is multiplied by another term or derivative, the equation is **nonlinear**; otherwise it is **linear**. An equation involving derivatives with respect to *more than one* independent variable is a **partial differential equation**. The simplest continuous-system models are linear, constant-coefficient ODEs, which can often be solved analytically; simulation becomes necessary once nonlinearities make an analytical solution impractical — the usual teaching route is therefore to build the simulation method on the linear case first and then extend it to nonlinear systems.

Continuous system simulation

A computer model of a physical system that continuously tracks the system's response according to a set of governing equations, typically differential equations. It is used to predict, e.g., rocket trajectories, explosive/blast dynamics, electric-circuit behaviour, human respiration, and robot motion.

The mechanical (mass–spring–damper) model. Consider a mass M restrained by a spring of stiffness K and a damper of coefficient D , driven by an input $F(t)$ scaled by K . This gives the second-order linear ODE that recurs through the rest of the chapter:

$$M\ddot{x} + D\dot{x} + Kx = KF(t), \quad (1)$$

where $x(t)$ is displacement, $\dot{x} = dx/dt$, $\ddot{x} = d^2x/dt^2$, and M, D, K are constant coefficients. (The same equation, with x reinterpreted as charge or current, describes an analogous *RLC* circuit — see the mechanical–electrical analogy of Chapter 2; it is not repeated here.)

Free response and damping classification. Setting $F(t) = 0$ gives the homogeneous equation $M\ddot{x} + D\dot{x} + Kx = 0$. Trying a solution $x = e^{st}$ and substituting gives the characteristic equation

$$Ms^2 + Ds + K = 0 \implies s = \frac{-D \pm \sqrt{D^2 - 4MK}}{2M}.$$

The sign of the discriminant $D^2 - 4MK$ fixes the character of the free response:

Overdamped ($D^2 > 4MK$): two distinct real negative roots; $x(t)$ decays to equilibrium with no oscillation.

Critically damped ($D^2 = 4MK$): a repeated real root $s = -D/2M$; the fastest decay obtainable without overshoot.

Underdamped / oscillatory ($D^2 < 4MK$): complex conjugate roots $s = -\frac{D}{2M} \pm j\omega_d$; $x(t)$ oscillates inside an exponentially decaying envelope before settling.

Example: a car suspension — too little damping ($D^2 < 4MK$) gives a bouncy, oscillatory ride; too much ($D^2 > 4MK$) gives a sluggish return to rest; designs usually target near-critical damping. This classification is used again in §3.3 to read the analog solution of (1), and in §3.6 to explain the parameter sweep of D in the CSMP program.

Distributed lag models. Not every continuous-system model is differential. A **distributed lag model** changes only at fixed time intervals (e.g. a year) and expresses each variable's current value from current *and previously lagged* values — it consists of linear algebraic, not differential, equations, and is common in econometrics. *Example:* a national-income model with consumption C , investment I , taxes T , government spending G and income Y : a static form $C = 20 + 0.7(Y - T)$, $I = 2 + 0.1Y$, $T = 0.2Y$, $Y = C + I + G$ is made dynamic by lagging the right-hand sides by one interval (subscript -1), e.g. $I = 2 + 0.1Y_{-1}$. With four equations in five unknowns, one variable (typically G) must be supplied externally each interval; the model is then solved recursively, interval by interval, from a set of starting values.

3.2 Analog Computer

Analog computer

A computation device that uses continuously variable physical quantities — electrical voltage, mechanical motion, hydraulic pressure — to model and solve mathematical problems. The most common form is the **electronic analog computer**, built from high-gain DC **operational amplifiers**. A voltage is equated to a model variable; amplifiers wired appropriately can **add** and **integrate** these voltages, and any voltage may be positive or negative according to the sign of the variable it represents.

Four building blocks are wired together to "program" a differential equation on an analog computer:



Figure 7: The four analog building blocks used to programme a differential equation.

- **Scale factor (potentiometer)** — a circle on a signal line; multiplies the variable by a constant coefficient of the model (e.g. K , D , $1/M$).
- **Summer** — a plain triangle; several scaled inputs enter its flat side and their sum leaves the apex.
- **Integrator** — a triangle with a bar on its input side; its output is the running time integral of the (summed) input.
- **Sign changer / inverter** — a triangle marked "−"; its output is the negative of its input, used wherever the model's sign convention requires it.

Accuracy. Three factors limit an electronic analog computer to roughly 0.1% accuracy: a voltage cannot be measured much more precisely than this; the idealised amplifier equations (notably, exactly zero output for zero input) are never perfectly true; and amplifiers have a limited output range, so every variable must be scaled to stay inside it. A digital computer is not subject to these limits, but integration is not a natural digital operation and must be approximated numerically (to high accuracy, given the right method).

	Analog computer	Digital computer
Representation	continuous physical quantity (voltage, motion, pressure)	discrete numeric values
Accuracy	limited, $\approx 0.1\%$ (measurement, amplifier assumptions, output range)	essentially unlimited (floating point)
Solving equations	simultaneously, in true parallel	one at a time, appears simultaneous by interleaving
Integration	natural (integrator circuit)	not natural; done by numerical approximation
Scope	usually dedicated to one problem at a time (time-sharing of sections is possible)	general purpose, easily reused for many problems

Analog computers remain attractive precisely because the wiring diagram mirrors the system's structure directly (simplifying both set-up and interpretation), and because it can be faster under the "true simultaneity" advantage above; its limited accuracy and single-problem dedication may simply not matter for a given application.

3.3 Analog Methods

The general analog-programming method is demonstrated on equation (1), $M\ddot{x} + D\dot{x} + Kx = KF(t)$. The standard first step is always to isolate the *highest derivative*:

$$M\ddot{x} = KF(t) - D\dot{x} - Kx. \quad (2)$$

Step-by-step wiring.

1. Assume, for now, that voltages representing $-x$ and $-\dot{x}$ are already available — they will actually be produced by the diagram itself, closing the loop.
2. Scale the three quantities $F(t)$, $-x$, $-\dot{x}$ by their coefficients K , K , D and add them with a summer: the output is $KF(t) - Kx - D\dot{x} = M\ddot{x}$, i.e. equation (2).
3. Scale $M\ddot{x}$ by $1/M$ and integrate: the output is $\dot{x}$.
4. Change the sign of $\dot{x}$ to get $-\dot{x}$ — supplying one of the assumed feedback signals — then integrate again to get $-x$, the other assumed signal.
5. Change the sign of $-x$ once more to present $+x(t)$ as the final output.

Feeding $-\dot{x}$ (scaled by D) and $-x$ (scaled by K) back to the summer's inputs closes the loop consistently with the assumption made in step 1:

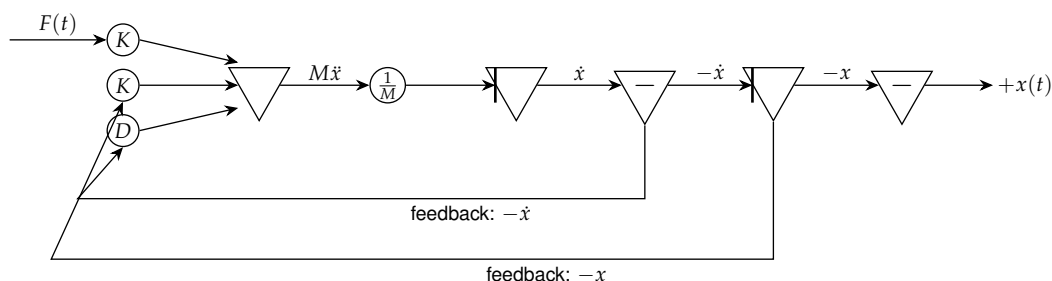


Figure 8: Analog block diagram solving $M\ddot{x} + D\dot{x} + Kx = KF(t)$ — automobile-suspension problem. A summer, scale factors, two integrators and two sign changers reconstruct $x(t)$ from $F(t)$.

The diagram is not unique — it can be redrawn differently depending on which variables are of interest and on the size of the scale factors used — and when a model has more than one independent variable, a separate diagram (with its own integrators) is drawn for *each* variable and the diagrams are cross-connected.

Example: the three-compartment liver model. Thyroxine injected into the blood is carried to the liver, converted there into iodine and absorbed into the bile, with some thyroxine re-circulated back to the blood. With x_1, x_2, x_3 the amounts in the blood, liver and bile compartments and transfer rates k_{12}, k_{21}, k_{23} proportional to concentration,

$$\dot{x}_1 = -k_{12}x_1 + k_{21}x_2, \quad \dot{x}_2 = k_{12}x_1 - (k_{21} + k_{23})x_2, \quad \dot{x}_3 = k_{23}x_2.$$

Three integrators are used, one per variable; interconnections between them, with sign changers where needed, supply each integrator's input exactly as the two-loop suspension diagram above supplies $\dot{x}$ and x to its own summer.

Solutions of the second-order equation. The classification of §3.1 shows directly in the analog output $x(t)$ for a step input:

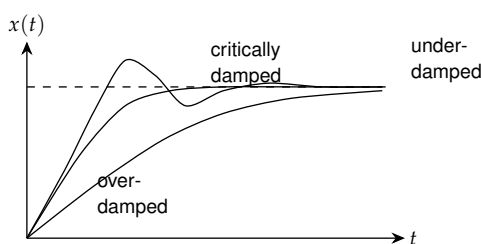


Figure 9: Family of solutions of $M\ddot{x} + D\dot{x} + Kx = KF(t)$ for a constant $F(t)$, by damping case. Lower damping gives more overshoot past the steady-state asymptote and more oscillation before settling; critical damping reaches the asymptote fastest without overshoot; overdamped responses approach it slowly and monotonically.

3.4 Hybrid Simulation

Hybrid simulation

Hybrid simulation combines analog and digital computation: a continuous simulation clock is maintained while updates to the model are event-driven, as in a discrete-event simulation.

It takes different forms depending on the application: one computer may simulate the system under study while the other simulates the *environment* in which it operates; or the system itself may already be an interconnection of continuous and discrete subsystems, naturally split between the two machines. Either way, high-speed **converters** are needed to transform signals between the analog (continuous) and digital (discrete) representations wherever the two sides interface.

3.5 Digital-Analog Simulators

Digital analog simulation (DAS)

A **programming technique** that makes a digital computer operate much like an analog one. A digital-analog simulator lets a continuous model be programmed on a digital computer using **macro-instructions** that carry out the action of the analog building blocks — adders, integrators, sign changers — and a program simply links these macros together, the software equivalent of wiring physical analog components.

This is the conceptual bridge to the fuller, statement-based languages of §3.6.

3.6 Continuous System Simulation Languages (CSSLs)

CSSL

A **Continuous System Simulation Language** accepts statement-type input, letting a problem be programmed directly from the equations of the mathematical model rather than by wiring hardware. A CSSL supplies macros and subroutines performing the function of the analog elements (summer,

integrator, sign changer, ...), lets the user define special-purpose elements for a given application, and includes a range of algebraic and logical expressions.

One widely used CSSL is **CSMP III** (Continuous System Modeling Program, Version III), a FORTRAN-like language. A program consists of three kinds of statement:

- **Structural statements** define the model, using FORTRAN's arithmetic notation (+ − * / **) and functional blocks for operations that occur repeatedly. *Example:* the equation $X = 6Y/W + (Z - 2)^2$ is coded as `X = 6.0*Y/W + (Z-2.0)**2.0`.
- **Data statements** assign numerical values to parameters, constants and initial conditions; `INCON` in particular sets an integrator's initial value.
- **Control statements** govern assembly, execution and output format — e.g. `CONST` (fixed value for a run), `PARAM` (a series of values, run once per value — a parametric sweep), `PRINT/PRTPLT` (tabulated/plotted output), `TIMER` (step size `DELT`, run length `FINTIM`, print/plot intervals `PRDEL/OUTDEL`), `TITLE/LABEL` (headings), and `END`.

Besides `INTGRL`, CSMP III supplies about a dozen further functional blocks mirroring common FORTRAN library routines — e.g. `LIMIT`, `STEP`, `EXP`, `ALOG`, `SIN`, `COS`, `SQRT`, `ABS`, `AMAX1`, `AMIN1` — so that a model is written almost entirely as statements, with no hardware wiring at all.

Example: the automobile-suspension ODE of (2), $M\ddot{x} = KF - Kx - D\dot{x}$, coded directly in CSMP III:

CSMP III statement	Meaning
<code>TITLE AUTOMOBILE SUSPENSION SYSTEM</code>	heading for the printed output
<code>PARAM D = (5.656, 16.968, 39.592, 56.56, 113.12)</code>	sweep: run once for each listed damping value D
<code>X2DOT = (1.0/M)*(K*F - K*X - D*XDOT)</code>	structural statement coding equation (2), $\ddot{x} = (1/M)(KF - Kx - D\dot{x})$
<code>XDOT = INTGRL(0.0, X2DOT)</code>	integrator: $\dot{x} = \int \ddot{x} dt$, initial value 0.0
<code>X = INTGRL(0.0, XDOT)</code>	integrator: $x = \int \dot{x} dt$, initial value 0.0
<code>CONST M=2.0, F=1.0, K=400.0</code>	fixed parameters for the run
<code>TIMER DELT=0.005, FINTIM=1.5, PRDEL=0.05, OUTDEL=0.05</code>	step size, run length, print and plot intervals
<code>PRINT X, XDOT, X2DOT</code>	tabulate the three variables
<code>PRTPLT X</code>	print-plot $x(t)$
<code>END</code>	end of the statements

This is the same two-integrator structure as Figure 2, written as statements instead of wired blocks — `X2DOT` is the summer/scale-factor output $M\ddot{x}$ rearranged, and each `INTGRL` is one integrator block. (Note $D = 56.56$ sits almost exactly at the critical value $D_c = 2\sqrt{MK} = 2\sqrt{800} \approx 56.57$ for this $M = 2, K = 400$ system, so the sweep spans under- through over-damped behaviour either side of it.)

3.7 Feedback Systems

Feedback system

A system that takes feedback from its output — the input is coupled with the output, so information about the result of the system's action is used to influence its next action.

In the general **negative-feedback control loop**, the output is measured and fed back to be combined (subtracted) with the input, producing an **error** signal that drives the controller/plant; a system with no such feedback path is **open loop**, and one with it is **closed loop**.

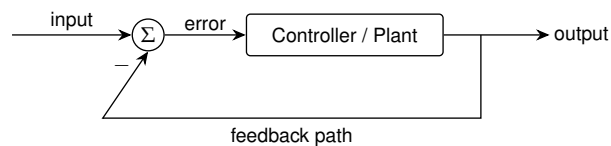


Figure 10: General negative-feedback control loop: the output is fed back and subtracted from the input to form the error that drives the controller/plant.

Example: a home heating system: the furnace heats the room, room temperature is the output, and the thermostat compares it with the set point and switches the furnace on or off accordingly — a simple two-state (bang-bang) closed-loop controller, rather than a continuously variable one.

The liver model of §3.3 is itself an example of feedback at the level of the model's structure: the pathway k_{21} carries compartment 2's output (liver) back into compartment 1's equation (blood), so x_1 's rate of change depends on x_2 , which in turn depends on x_1 — a closed loop internal to the system being simulated, distinct from a control loop imposed around it.

Chapter 4 Simulation of Queuing Systems

4.1 Elements of a Queuing System

Queuing system

A **queuing system** consists of one or more servers that provide service of some kind to arriving customers. If a server is busy, an arriving customer joins one or more queues formed in front of the servers. Queuing theory studies how such waiting lines behave, so that waiting time and server idle time can be balanced to keep total cost at a minimum.

Waiting lines arise from a mismatch between demand and service capacity: too much demand makes customers wait; too little demand leaves the server idle. Everyday waiting lines include people at a railway ticket window, vehicles at a petrol pump or a traffic signal, products at a machining centre, or television sets left at a repair shop – the objective in every case is to balance the cost of customers waiting against the cost of keeping servers idle, so that total cost is minimised. Rising server capacity cuts waiting cost but raises idle-server cost, and vice versa; queuing analysis exists to locate that balance quantitatively rather than by guesswork. The mathematical study of queues is credited to A. K. Erlang, who in 1920 analysed waiting lines of telephone calls in Copenhagen, where operators could not handle all calls during busy periods. The same structure recurs in many real systems.

System	Servers	Customers
Bank	Tellers	Customers
Hospital	Doctors, nurses, beds	Patients
Computer system	CPU, I/O devices	Jobs
Manufacturing	Machines, workers	Parts

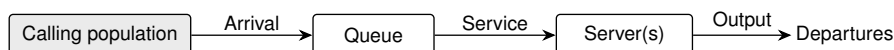


Figure 11: Generic queuing system: population → arrival → queue → service → output.

Every arrival is resolved by a simple rule: if the server is free the customer enters service immediately; if the server is busy the customer joins the queue.

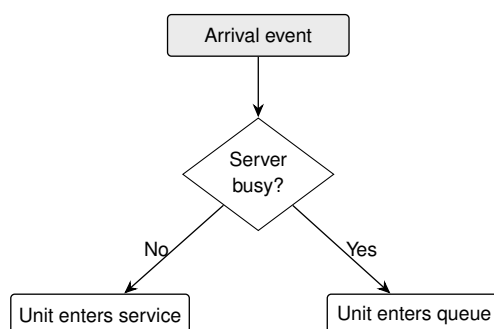


Figure 12: Decision on each arrival: enter service if the server is free, otherwise join the queue.

A queuing system has five elements, read off Figure 1 left to right:

1. **Population of customers** (calling source) – the pool from which customers are drawn. It may be **limited** (a fixed number of processes waiting to run, machines waiting for one repairman) or **unlimited** (a bank on a busy street). “Customer” is used generally: a person, machine, computer process or telephone call.

2. **Arrival** – the way customers enter the system, usually random, described by the pattern (distribution) of the interval between two successive arrivals.
3. **Queue** (waiting line) – the customers waiting for service; the customer currently being served is *not* counted in the queue. A queue has a **maximum size** (= system capacity: customers waiting plus the one(s) in service; arrivals beyond it are lost) and a **queuing discipline** (Section 4.2).
4. **Service** – the activity that takes time. It may be a literal service or an abstraction (a CPU time slice, a call connection).
5. **Output** – how a customer leaves the system. Usually ignored, but in some systems (e.g. round-robin CPU scheduling) a customer leaving the server re-enters the queue.

4.2 Characteristics of a Queuing System

A queuing system is fully specified by six characteristics, which overlap with the elements above but are the more exam-relevant framing:

Characteristic	Description
Calling population	Finite (closed system – arrival rate depends on how many customers are already in service/queue, e.g. a fixed set of jobs on a computer) or infinite (open system – arrival rate unaffected by customers already served, e.g. a bank).
Arrival pattern	Usually random; commonly a Poisson arrival process (restaurants, call centres, demand for a product). Arrivals may also be scheduled (clinic appointments, flights), single or in batches.
System capacity	The maximum number of customers allowed in queue + service. Most models assume a limit in practice, though some theoretical models take it as unlimited.
Queue behaviour & discipline	See below.
Service	Real or abstract; the service time is typically random, commonly exponentially distributed.
Output	How customers leave; occasionally customers re-enter the queue (round robin).

Poisson arrival process

Arrivals form a **Poisson process** of rate λ (customers per unit time) when the number of arrivals in any interval of length t has probability

$$P(n \text{ arrivals in time } t) = \frac{e^{-\lambda t} (\lambda t)^n}{n!}, \quad n = 0, 1, 2, \dots$$

Equivalently, the time between two successive arrivals (the interarrival time) is **exponentially** distributed with mean $1/\lambda$ and is **memoryless**: how long one has already waited tells nothing about how much longer until the next arrival. This is the “M” in Kendall’s notation (Section 4.5).

Example: With $\lambda = 4$ customers/hour, the chance of no arrival in a 15-minute interval ($t = 0.25$ hr) is $P(0) = e^{-4(0.25)} (4 \times 0.25)^0 / 0! = e^{-1} \approx 0.368$.

Queue behaviour describes what customers do while waiting: **balk** (leave on seeing the line is too long, before joining), **renege** (join, then leave because the line moves too slowly), **jockey** (switch queues, believing another is faster).

Queuing discipline decides which waiting customer is served next when a server frees up:

- **FIFO/FCFS** – First In First Out / First Come First Served (an “orderly” queue).
- **LIFO/LCFS** – Last In First Out / Last Come First Served (behaves like a stack).

- **SIRO** – Serve In Random Order.
- **SPT** – Shortest Processing Time first.
- **PR** – service according to Priority.

Unless stated otherwise, FIFO is assumed.

4.3 Model of a Queuing System

Queuing systems are modelled by how servers are arranged relative to the queue(s):

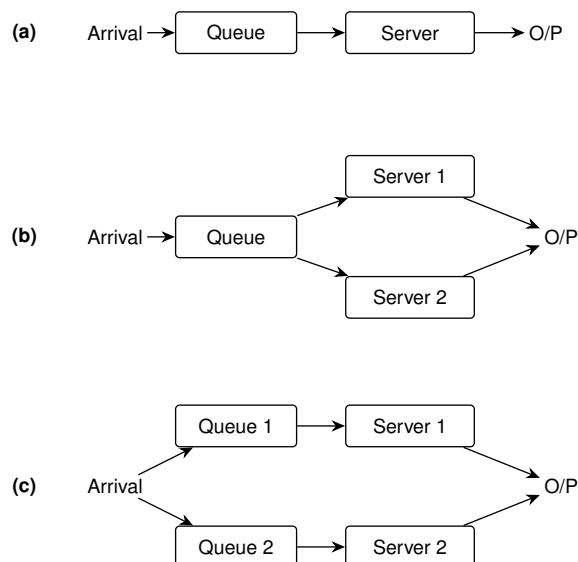


Figure 13: (a) Single-server system; (b) single-queue, multiple-server system (one shared queue feeds n parallel servers); (c) multiple-queue, multiple-server system (each server has its own dedicated queue).

- **Single server queue system** – one queue, one server. Notation $M/M/1/K \cong M/M/1$ when capacity K is written explicitly.
- **Single queue, multiple server system** ($M/M/n$) – one shared queue feeds n parallel servers; whichever server frees up next takes the head-of-line customer. Example: a bank with one line feeding several tellers.
- **Multiple queue, multiple server system** – each server has its own dedicated queue (e.g. separate supermarket checkout lanes, each with its own cashier); effectively n independent single-server systems in parallel, so each can be analysed on its own with the single-server formulas of Section 4.6.

In Kendall's notation these correspond to $M/M/1$ (single server), $M/M/n$ (single queue, n servers) and $n \times (M/M/1)$ (multiple queue, multiple server), when arrivals and service are Poisson/exponential.

4.4 Types of Queuing Systems

Queuing systems can also be classified by the physical/behavioural layout of the queue itself:

- **Structured queue** – forms in a fixed, predictable layout: supermarket checkouts, bank counters, airport security lines.
- **Unstructured queue** – forms in unpredictable locations/directions, typical of taxi ranks, ATMs and periods of high demand.
- **Kiosk-based queue** – common at medical, banking and government offices: arriving customers enter basic details (and reason for visit) into a kiosk ("take a number"), and this information is used to route them for faster service.

Type	Typical setting	Notable feature
Structured	Bank counters, airport security	Fixed physical layout, easy to model as a single line
Unstructured	Taxi ranks, ATMs, peak-hour crowds	No fixed lane; position in line is informal
Kiosk-based	Hospitals, government offices	Waiting is virtual (a number); customers need not stand in line

This is a separate classification axis from Section 4.3: a real system can combine both, e.g. a bank (structured) with a single queue feeding several tellers ($M/M/n$), or a taxi rank (unstructured) served by one driver at a time ($M/M/1$). A further classification, by how servers are chained rather than how the queue looks, is **series (tandem) queues**, where a customer departing one queue joins another downstream – treated in Section 4.7 (Network of Queues).

4.5 Queuing Notation (Kendall's Notation)

Kendall's notation

Kendall (1953) proposed a shorthand for classifying a queuing system by six symbols:

$$A / B / c / N / K / D$$

Symbol	Meaning
A	Arrival-time (interarrival) distribution.
B	Service-time distribution.
c	Number of (parallel) servers.
N	System capacity (maximum in queue + service). Omitted if unlimited.
K	Size of the calling population. Omitted if unlimited (infinite population).
D	Queuing discipline (FIFO, LIFO, SIRO, ...). Omitted if FIFO (the default).

Trailing symbols may be dropped once they take default values (unlimited N , unlimited K , FIFO D); e.g. $M/M/1$ is short for $M/M/1/\infty/\infty/\text{FIFO}$. Some texts instead order the same six slots as servers-discipline-capacity-population; whichever order is used, the six meanings above are unchanged.

Standard codes used for the distribution slots A and B :

Code	Distribution
M	Markovian/Poisson – exponential interarrival times or exponential service times.
E_k	Erlang- k distribution of intervals or service duration.
D	Deterministic – constant, known interarrival time or service time (no randomness).
G	General – arbitrary, unspecified distribution.

(The D here, inside slot A or B , means *deterministic distribution*; it is unrelated to the discipline slot D at the end of the six-symbol notation.)

The Erlang- k code E_k models a service (or interarrival) time built from k independent, identically distributed exponential phases in series; it bridges the gap between purely random ($E_1 = M$) and constant ($E_\infty \rightarrow D$) durations, and is chosen when real data is less variable than exponential but still random.

The value of fixing this notation is that each combination of A and B determines which formulas can be used: $M/M/1$ and $M/M/c$ have simple closed-form L , L_q , W , W_q (Section 4.6); $M/G/1$ needs the more involved Pollaczek–Khinchine formula; and general $G/G/1$ systems usually have no closed form

and must be studied by simulation – which is precisely why this course treats queuing systems as simulation models rather than only as algebra.

Example: Decoding notations.

Notation	Reading
$M/M/1$	Poisson arrivals, exponential service, 1 server, FIFO, unlimited capacity and population – the running example of this chapter.
$M/M/c$	Poisson arrivals, exponential service, c parallel servers, FIFO, unlimited capacity and population.
$D/M/1$	Deterministic (known) arrivals, exponential service, 1 server, FIFO, unlimited capacity and population.
$M/G/3/20$	Poisson arrivals, general service time, 3 servers, system capacity 20, unlimited population.
$M/M/3/FCFS/20/1500$	Poisson arrivals and service, 3 servers, FCFS, capacity 20 (17 waiting + 3 in service), population of 1500 jobs.
$M/D/6/20/1000/LIFO$	Poisson arrivals, deterministic service, 6 servers, capacity 20, population 1000, LIFO discipline.
$D/M/6/FIFO/25/3000$	Deterministic arrivals, exponential service, 6 servers, FIFO, capacity 25, population 3000.

4.6 Measurement of System Performance

Four response parameters are generally used to evaluate a queuing system: (1) average number of customers in the queue/system, (2) average waiting time of customers in the queue/system, (3) system (server) utilization, and (4) the cost of waiting time and idle time. The average number in queue indicates space needed for waiting customers – too long a queue can discourage new customers, while no queue at all may signal poor service quality. The average waiting time is needed to cost the time customers spend waiting; together the four parameters let a manager judge whether current capacity should be increased, decreased or left unchanged.

Symbol	Meaning
λ	Mean arrival rate (customers per unit time).
μ	Mean service rate per server (customers per unit time).
ρ (or s)	Utilization factor, λ/μ for one server or $\lambda/(\mu n)$ for n servers.
L	Expected number of customers in the whole system (queue + in service).
L_q	Expected number of customers in the queue only (excludes the one in service).
W	Expected time a customer spends in the whole system (waiting + service).
W_q	Expected time a customer spends waiting in the queue only.

System (server) utilization, denoted ρ (or s), is the fraction of capacity actually used:

$$\text{single server: } \rho = \frac{\lambda}{\mu}, \quad n \text{ servers: } \rho = \frac{\lambda}{\mu n},$$

where λ is the mean arrival rate and μ the mean service rate per server. Raising λ raises ρ but also raises queue length and waiting time without bound as $\rho \rightarrow 1$, so 100% utilization is not a realistic operating target.

Little's Law

In steady state, the average number of customers in a system equals the average arrival rate times

the average time a customer spends in that system:

$$L = \lambda W, \quad L_q = \lambda W_q.$$

It applies to the system as a whole (L, W) or to the queue alone (L_q, W_q), and needs no assumption on the arrival or service distributions.

The M/M/1 queue

The M/M/1 queue is the most common queuing model, used e.g. for a single processor or a single device in a computer system.

M/M/1 assumptions

- Interarrival times are exponentially distributed with mean arrival rate λ (Poisson arrivals).
- Service times are exponentially distributed with mean service rate μ .
- A single server.
- FCFS (FIFO) queue discipline.
- Unlimited queue length (infinite capacity).
- Infinite (unlimited) calling population.
- Stability requires $\lambda < \mu$, i.e. $\rho = \lambda/\mu < 1$.

Because arrivals and services are memoryless, the number of customers in the system behaves as a birth–death process: it increases by one at rate λ and decreases by one at rate μ . Balancing the flow into and out of each state n gives $P_n = \rho^n P_0$, and since probabilities sum to 1, $P_0 = 1 - \rho$ and $P_n = (1 - \rho)\rho^n$; summing $n P_n$ over all n then yields L below.

Example: For $\rho = 0.6$: $P_0 = 1 - 0.6 = 0.4$ (system empty), $P_1 = (1 - 0.6)(0.6)^1 = 0.4 \times 0.6 = 0.24$, $P_2 = (1 - 0.6)(0.6)^2 = 0.4 \times 0.36 = 0.144$ – the probability of finding exactly n customers falls off geometrically as n grows.

Its steady-state performance measures are:

$$L = \frac{\lambda}{\mu - \lambda}, \quad L_q = \frac{\lambda^2}{\mu(\mu - \lambda)}, \quad W = \frac{L}{\lambda} = \frac{1}{\mu - \lambda}, \quad W_q = W - \frac{1}{\mu} = \frac{\lambda}{\mu(\mu - \lambda)}.$$

$W = L/\lambda$ and $W_q = L_q/\lambda$ are just Little's Law solved for W ; $W_q = W - 1/\mu$ removes the mean service time $1/\mu$ from the total time in system.

For c identical parallel servers (M/M/c), the same balance-equation approach applies, but with $\rho = \lambda/(c\mu)$ and a summation over the c states in which not all servers are busy; the resulting L, L_q, W, W_q (the Erlang-C formulas) are algebraically heavier and are not required here – only the utilization $\rho = \lambda/(c\mu)$ from Section 4.6 is needed for M/M/c at this level.

M/M/1 summary. $\rho = \lambda/\mu < 1$ for stability. $L = \rho/(1 - \rho) = \lambda/(\mu - \lambda)$; $L_q = \rho L = \lambda^2/(\mu(\mu - \lambda))$; $W = L/\lambda$; $W_q = W - 1/\mu = L_q/\lambda$. As $\rho \rightarrow 1$, $L, L_q, W, W_q \rightarrow \infty$.

Example: Comparing two candidate servers. Customers arrive at rate $\lambda = 10$ /hour. Management must choose between Server 1 ($\mu_1 = 11$ /hour) and Server 2 ($\mu_2 = 12$ /hour). Compute ρ, L, L_q, W, W_q for each and recommend one.

Quantity	Formula	Server 1 ($\mu_1 = 11$)	Server 2 ($\mu_2 = 12$)
ρ	λ/μ	$10/11 = 0.909$	$10/12 = 0.833$
L	$\lambda/(\mu - \lambda)$	$10/(11 - 10) = 10/1 = 10$	$10/(12 - 10) = 10/2 = 5$
L_q	$\lambda^2/(\mu(\mu - \lambda))$	$10^2/(11 \times 1) = 100/11 = 9.09$	$10^2/(12 \times 2) = 100/24 = 4.16$
W	L/λ	$10/10 = 1 \text{ hr}$	$5/10 = 0.5 \text{ hr}$
W_q	$W - 1/\mu$	$1 - 1/11 = 1 - 0.0909 = 0.90 \text{ hr}$	$0.5 - 1/12 = 0.5 - 0.0833 = 0.4167 \text{ hr}$

Server 2 gives a lower number in system, a lower waiting time, and only a slightly lower utilization than Server 1 – so **Server 2 ($\mu_2 = 12/\text{hour}$) is the better choice**: it serves customers faster without leaving the server significantly more idle, which also makes the system more attractive to new customers. This is the waiting-cost/idle-cost balance of Section 4.1 in numbers: Server 2 likely costs more to provide (a faster server or worker), but that cost is weighed against roughly halving both L and W_q – a concrete, quantified trade-off rather than a guess.

Example: Sensitivity to the arrival rate. Fix $\mu = 10/\text{hour}$ and raise λ from 5 in steps of 20% (5, 6, 7.2, 8.64) up to $\lambda = \mu = 10$, using $\rho = \lambda/\mu$, $L = \lambda/(\mu - \lambda)$, $W = L/\lambda$. Worked in full for $\lambda = 8.64$:

$$\rho = \frac{8.64}{10} = 0.864, \quad L = \frac{8.64}{10 - 8.64} = \frac{8.64}{1.36} = 6.35, \quad W = \frac{6.35}{8.64} = 0.73 \text{ hr}.$$

The remaining rows follow the same substitutions:

λ	5.0	6.0	7.2	8.64	10.0
$\rho = \lambda/\mu$	0.500	0.600	0.720	0.864	1.000
$L = \lambda/(\mu - \lambda)$	1.00	1.50	2.57	6.35	∞
$W = L/\lambda$	0.20	0.25	0.36	0.73	∞

As $\lambda \rightarrow \mu$ (i.e. $\rho \rightarrow 1$), both L and W diverge – a small increase in arrival rate near saturation causes a disproportionately large increase in congestion, confirming why 100% utilization is never targeted in practice.

4.7 Network of Queues

Many real systems are naturally modelled as a **network of queues**: customers departing one queue are routed into another, rather than a single isolated queue being analysed on its own. A network is **open** if customers enter from, and eventually leave to, outside the system (the usual case below), and **closed** if a fixed number of customers circulate within the network indefinitely (e.g. a fixed set of jobs cycling between a CPU and I/O devices).

Example: Discount warehouse (fork-join network). Arriving customers split into two front-end stations: a self-service area (Service Centre 1, $c = \infty$) or one of three clerks (Service Centre 2, $c = 3$). Both streams then feed a single downstream queue for one cashier (Service Centre 3, $c = 1$), who checks everyone out before they depart. Two parallel front stations join into one shared checkout queue – a fork-join network.

Example: Candy production line (series/tandem network). Three single-server stations in series, each with a finite in-process buffer of capacity 1000 boxes between them; raw material at the first queue is assumed always available.

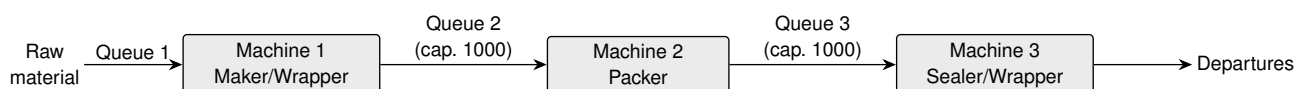


Figure 14: Three single-server stations in series (tandem network) with finite in-process buffers.

If the buffer after Machine 1 (Queue 2) fills up, Machine 1 is **blocked** and must stop; if that buffer empties, Machine 2 is **starved** and must stop – both machines' throughput is coupled through the

shared buffer capacities, which is the key modelling difference from analysing each machine as an isolated queue.

When a queue's output splits between two downstream stations, or several queues merge into one (as Centres 1 and 2 merge into Queue 3 in the warehouse example), the arrival rate entering a downstream queue is the corresponding sum or fraction of the upstream rates. Little's Law ($L = \lambda W$) still holds node by node for each queue in the network, and the expected number of customers in the whole network is simply the sum of the L values of its individual queues – so a network of queues can be analysed piece by piece rather than all at once, provided each piece is itself a standard queue (e.g. $M/M/1$ or $M/M/c$).

4.8 Applications of Queuing Systems

Queuing models are used wherever entities compete for a limited number of servers and management must decide on capacity – how many tellers, checkouts, machines or channels to provide – without either starving customers of service or over-staffing an idle facility. The elements, characteristics, models and formulas of this chapter apply, with different labels for “customer” and “server”, across very different industries:

- **Telecommunications** – call centres, telephone exchanges, packet-switched networks.
- **Banks** – teller counters, ATM queues.
- **Traffic control** – vehicles at signals, toll booths, petrol pumps.
- **Computer systems** – CPU and I/O job scheduling, determining the sequence of computer operations.
- **Health services** – hospital bed assignment, doctor/nurse scheduling, patient queues.
- **Airports** – runway traffic, airline ticket sales, security lines.
- **Manufacturing** – layout of production systems, machine and worker scheduling (e.g. the candy production line above).

Chapter 4 recap. A queuing system = population → arrival → queue → service → output, governed by a queuing discipline (FIFO, LIFO, SIRO, SPT, PR). Kendall's notation $A/B/c/N/K/D$ summarises the arrival and service distributions, number of servers, system capacity, population size and discipline. For $M/M/1$: $\rho = \lambda/\mu$, $L = \lambda/(\mu - \lambda)$, $L_q = \lambda^2/(\mu(\mu - \lambda))$, $W = L/\lambda$, $W_q = W - 1/\mu$, and Little's Law $L = \lambda W$ links system and queue measures in any stable network of queues.

Chapter 5 Markov Chains

5.1 Key Features of Markov Chains

A **stochastic process** is a collection of random variables $\{X_0, X_1, X_2, \dots\}$ indexed by time (or step number n), where X_n is the state of the system at step n . Each X_n takes a value from the **state space**, the set of all possible **states** (outcomes/conditions) the system can be in — for example {Coke, Pepsi} for a brand-switching process, or {rain, no rain} for a weather process. This chapter deals only with a **discrete-time, discrete-state** process: time advances in steps $n = 0, 1, 2, \dots$ (e.g. one purchase, one day) and the state space is a finite list of categories, not a continuous quantity. A random walk in which a walker moves between locations, each location being a state and restricted from returning to the location just visited, is a classic illustrative case of such a chain.

If future states depend on the past only through the present state, the process is a **Markov process**; a Markov process with a discrete state space is precisely what is called a **Markov chain**. This dependency structure is easiest to see as a simple left-to-right chain of states, each one built only from its immediate predecessor:

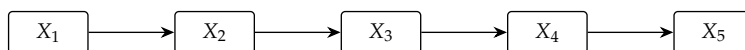


Figure 15: Each state depends only on the one immediately before it – the Markov dependency chain.

Markov property (memorylessness)

A stochastic process $\{X_n\}$ is a **Markov chain** if, for all states and all n ,

$$P(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = P(X_{n+1} = j \mid X_n = i) = p_{ij}.$$

In words: the probability distribution of the *next* state depends only on the *current* state, and not on the sequence of states that preceded it — the chain has no memory of how the present state was reached.

The conditional probability $p_{ij} = P(X_{n+1} = j \mid X_n = i)$ is called the **transition probability**: it is the probability that the system moves to state j given that it is currently in state i . Because it is a conditional probability, it always lies in $[0, 1]$.

A sequence of trials is a Markov chain iff three conditions hold together:

1. **Discrete state space** — the outcome of each trial is one of a finite (or countable) set of states.
2. **Markov property** — the outcome of a trial depends only on the present state, not on any earlier states.
3. **Stationarity (time-homogeneity)** — the transition probabilities p_{ij} stay the same from one transition to the next; they do not depend on the time index n . Such a chain is called a **stationary** (or homogeneous) Markov chain.

All chains treated in this course are stationary and **first order**: the next state depends on exactly one immediately preceding state (not on states two or more steps back).

Since exact future states cannot be predicted from a random system, what *can* be predicted are the statistical properties of the system's future — this is precisely what Markov chains capture, and it is why they are widely used for prediction (weather, stock prices, product success, and so on). Markov chains are also the natural model for queuing systems: in an $M/M/m$ queue, the number of jobs waiting in the queue is itself a Markov chain, and the time a single job spends in the system is a Markov process. All the chains worked in this chapter are **irreducible** (every state can eventually be reached from every other state) and **aperiodic** (there is no fixed cycle length forcing a return to a state), so they

are **ergodic**: a unique steady-state distribution exists and is reached from any starting distribution as $n \rightarrow \infty$ (Section 5.2).

5.2 Markov Process with Examples

The transition probability matrix

Collecting every one-step transition probability $p_{ij} = P(X_{n+1} = j \mid X_n = i)$ of an n -state chain into a square array gives the **transition probability matrix** (or **stochastic matrix**) P , with rows indexed by the current state and columns by the next state:

$$P = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & p_{22} & \cdots & p_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{pmatrix}.$$

Property of a stochastic matrix

Every row of P sums to 1: $\sum_j p_{ij} = 1$ for every state i . This must hold because, from any given state, the system must move to *some* state (possibly the same one) at the next step, so the row lists a complete probability distribution over next states.

One-step and n -step transitions

The one-step transition probability is $p_{ij} = \Pr(X_1 = j \mid X_0 = i)$. The **n -step transition probability** $p_{ij}^{(n)} = \Pr(X_n = j \mid X_0 = i)$ is the probability of being in state j after n steps, having started in state i . Because each intermediate step multiplies transition probabilities and every path to j must be summed over, the n -step transition matrix is simply the n -th power of the one-step matrix:

$$P^{(n)} = P^n.$$

This matrix-power result (a special case of the **Chapman–Kolmogorov** relation $P^{(m+n)} = P^{(m)}P^{(n)} = \sum_k p_{ik}^{(m)} p_{kj}^{(n)}$) is the main computational tool for this chapter: to find any n -step probability, raise P to the power n and read off the required entry. The reasoning behind it is simple path counting – to go from i to j in two steps the chain must pass through *some* intermediate state k , so $p_{ij}^{(2)} = \sum_k p_{ik} p_{kj}$, which is exactly the (i, j) entry of the matrix product $P \times P$; the same argument extended one more step gives $P^3 = P \times P^2$, and so on for any n .

Worked example: Coke vs. Pepsi brand switching

Example: Given that a person's last cola purchase was Coke, there is a 90% chance the next purchase is again Coke (so 10% switch to Pepsi). Given that the last purchase was Pepsi, there is an 80% chance the next purchase is again Pepsi (so 20% switch to Coke).

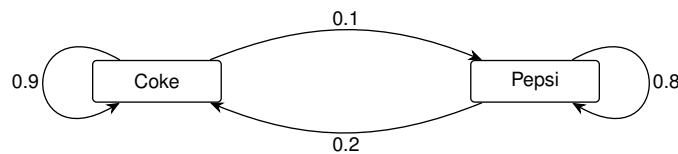


Figure 16: State-transition diagram for the Coke/Pepsi brand-switching chain.

With states ordered (Coke, Pepsi), the one-step transition matrix and its row-sum check are

$$P = \left(\begin{array}{c|cc} & \text{C} & \text{P} \\ \hline \text{C} & 0.9 & 0.1 \\ \text{P} & 0.2 & 0.8 \end{array} \right), \quad 0.9 + 0.1 = 1, \quad 0.2 + 0.8 = 1.$$

(a) Given a current Pepsi purchaser, find the probability of buying Coke two purchases from now, i.e. entry (P, C) of $P^2 = P \times P$. Every entry is a full dot product of a row of P with a column of P :

$$\begin{aligned} (C, C) &: 0.9(0.9) + 0.1(0.2) = 0.81 + 0.02 = 0.83 \\ (C, P) &: 0.9(0.1) + 0.1(0.8) = 0.09 + 0.08 = 0.17 \\ (P, C) &: 0.2(0.9) + 0.8(0.2) = 0.18 + 0.16 = 0.34 \\ (P, P) &: 0.2(0.1) + 0.8(0.8) = 0.02 + 0.64 = 0.66 \end{aligned} \quad \Rightarrow \quad P^2 = \begin{pmatrix} 0.83 & 0.17 \\ 0.34 & 0.66 \end{pmatrix}.$$

So $\Pr(\text{Pepsi} \rightarrow \text{Coke in 2 steps}) = 0.34 = 34\%$. (Check: the same value is obtained by direct path enumeration, $\Pr[P \rightarrow C \rightarrow C] + \Pr[P \rightarrow P \rightarrow C] = (0.2)(0.9) + (0.8)(0.2) = 0.18 + 0.16 = 0.34$.)

(b) Given a current Coke purchaser, find the probability of buying Pepsi after three purchases, i.e. entry (C, P) of $P^3 = P \times P^2$:

$$\begin{aligned} (C, C) &: 0.9(0.83) + 0.1(0.34) = 0.747 + 0.034 = 0.781 \\ (C, P) &: 0.9(0.17) + 0.1(0.66) = 0.153 + 0.066 = 0.219 \\ (P, C) &: 0.2(0.83) + 0.8(0.34) = 0.166 + 0.272 = 0.438 \\ (P, P) &: 0.2(0.17) + 0.8(0.66) = 0.034 + 0.528 = 0.562 \end{aligned} \quad \Rightarrow \quad P^3 = \begin{pmatrix} 0.781 & 0.219 \\ 0.438 & 0.562 \end{pmatrix}.$$

So $\Pr(\text{Coke} \rightarrow \text{Pepsi in 3 steps}) = 0.219 = 21.9\%$.

(c) Population forecast. Each person buys one cola per week; today 60% of people drink Coke and 40% drink Pepsi, i.e. the initial distribution is the row vector $\pi_0 = (0.6, 0.4)$. The distribution after n steps is $\pi_0 P^n$, so the Coke fraction after 3 weeks is π_0 dotted with the Coke column of P^3 :

$$\Pr[X_3 = \text{Coke}] = 0.6(0.781) + 0.4(0.438) = 0.4686 + 0.1752 = 0.6438 = 64.38\%.$$

The same stepwise method (state diagram $\rightarrow$ transition matrix $\rightarrow$ matrix powers $\rightarrow$ dot with π_0) applies unchanged to other two-state chains built the same way – only the matrix entries change. For instance,

a weather chain with states {rain, no rain} and $P = \begin{pmatrix} 0.4 & 0.6 \\ 0.2 & 0.8 \end{pmatrix}$ (rain persists 40% of the time, no-rain

persists 80% of the time) gives, by the identical entry-by-entry squaring, $P^2 = \begin{pmatrix} 0.28 & 0.72 \\ 0.24 & 0.76 \end{pmatrix}$; so starting from a dry day, the chance of rain the day after tomorrow is the (no rain, rain) entry, $0.24 = 24\%$.

Formula recap. Transition matrix: $P_{ij} = \Pr(X_{n+1} = j \mid X_n = i)$, rows sum to 1. n -step matrix: $P^{(n)} = P^n$ (Chapman–Kolmogorov). Distribution after n steps from initial split π_0 : $\pi_n = \pi_0 P^n$. Steady state: solve $\pi P = \pi$, $\sum_i \pi_i = 1$.

Steady-state (stationary) distribution

Steady-state distribution

A row vector $\pi = (\pi_1, \dots, \pi_n)$ is the **steady-state** (or **stationary**, or **limiting**) distribution of chain P if

$$\pi P = \pi, \quad \sum_i \pi_i = 1.$$

That is, π is unchanged by one more step of the chain (π is a left eigenvector of P for eigenvalue 1, normalised to sum to 1). Equivalently, as $n \rightarrow \infty$ every row of P^n converges to the same vector π , so the long-run state probabilities no longer depend on the starting state.

Example: Find the steady-state distribution of the Coke/Pepsi chain above, $P = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$. Let $\pi = (\pi_C, \pi_P)$. Writing out $\pi P = \pi$ column by column gives the **balance equations**, together with

normalisation:

$$0.9\pi_C + 0.2\pi_P = \pi_C \quad (\text{i, Coke column})$$

$$0.1\pi_C + 0.8\pi_P = \pi_P \quad (\text{ii, Pepsi column})$$

$$\pi_C + \pi_P = 1 \quad (\text{iii, normalisation})$$

From (i): $0.9\pi_C + 0.2\pi_P = \pi_C \Rightarrow 0.2\pi_P = 0.1\pi_C \Rightarrow \pi_P = 0.5\pi_C$. Substituting into (iii):

$$\pi_C + 0.5\pi_C = 1 \Rightarrow 1.5\pi_C = 1 \Rightarrow \pi_C = \frac{2}{3} \approx 0.6667, \quad \pi_P = 0.5\left(\frac{2}{3}\right) = \frac{1}{3} \approx 0.3333.$$

Check with (ii): $0.1(2/3) + 0.8(1/3) = 0.0667 + 0.2667 = 0.3333 = \pi_P$ – consistent. Check sum: $2/3 + 1/3 = 1$ – confirmed.

So in the long run, regardless of the starting split between Coke and Pepsi drinkers, the market settles at $\pi_C = 2/3 \approx 66.7\%$ Coke and $\pi_P = 1/3 \approx 33.3\%$ Pepsi.

Practice: The same technique solves, e.g., a Honda/Yamaha switching problem with $P = \begin{pmatrix} 0.7 & 0.3 \\ 0.5 & 0.5 \end{pmatrix}$

(Honda keeps 70% of its buyers, Yamaha keeps 50%). Computing P^2 and P^3 by the same entry-by-entry method gives $P^3 = \begin{pmatrix} 0.628 & 0.372 \\ 0.620 & 0.380 \end{pmatrix}$, so a current Honda buyer purchases Yamaha after three purchases with probability $0.372 = 37.2\%$; starting from a 50–50 split, 50% Honda after 3 purchases becomes $0.5(0.628) + 0.5(0.620) = 62.4\%$ Honda. Try reproducing these two matrix powers by hand as practice.

5.3 Applications of Markov Chains

Because a Markov chain predicts the *statistical* long-run behaviour of a system even when its exact future state is unpredictable, it is used wherever a system moves between a discrete set of states over time:

- **Weather forecasting** – predicting tomorrow’s weather (e.g. rain / no rain) from today’s, as in the worked example above.
- **Brand switching / market-share prediction** – forecasting how a population splits across competing products or services over time, exactly as computed for the Coke/Pepsi chain.
- **Stock market prediction** – modelling price or trend movements (up/down/unchanged) as transitions between discrete market states.
- **Forecasting product success** – tracking how customer adoption of a product evolves purchase to purchase.
- **Queuing theory** – the number of jobs waiting in an $M/M/m$ queue is itself a Markov chain, and the time a job spends in the system is a Markov process (link to Chapter 4).
- **Routing and the PageRank algorithm** – a web surfer’s next-page click depends only on the current page; ranking pages by their steady-state visit probability is exactly a Markov steady-state computation.
- **Pattern recognition** and studying **web navigation behaviour** of internet users – both treat the next observed symbol/page as depending only on the current one.

Chapter summary. A Markov chain is a discrete-state stochastic process with the memoryless (Markov) property and time-stationary transition probabilities p_{ij} , collected into the stochastic matrix P (rows summing to 1). The n -step transition matrix is P^n ; the distribution after n steps from an initial split π_0 is $\pi_0 P^n$; and the long-run steady-state distribution π solves $\pi P = \pi$ with $\sum_i \pi_i = 1$, independent of the starting state for an irreducible, aperiodic chain.

Chapter 6 Random Number Generation

6.1 Properties of Random Numbers

A **random number** is a value drawn from a uniformly distributed random variable over a specified interval, with every value in that interval equally likely to occur. Random numbers are the raw material of a stochastic simulation: they are generated on $[0, 1]$ and then transformed – by the inversion, rejection, composition or convolution techniques of Section 6.6 – into the interarrival times, service times, demand levels and other random inputs that drive the model. If the numbers feeding the model are not genuinely random-looking, every downstream result (queue lengths, waiting times, utilisation) inherits the same bias, so the quality of the generator directly determines the credibility of the simulation.

Two statistical properties of a random-number sequence

Each random number R_i must be an independent sample drawn from the continuous uniform distribution on $(0, 1)$,

$$f(x) = \begin{cases} 1, & 0 \leq x \leq 1 \\ 0, & \text{otherwise.} \end{cases}$$

Uniformity. Every sub-interval of $[0, 1]$ of equal length is equally likely to contain a generated value: if $[0, 1]$ is divided into n equal classes and a total of m numbers is generated, the expected number of observations falling in each class is m/n . Equivalently, over a long run the R_i should have sample mean close to 0.5 and sample variance close to $1/12$, the mean and variance of $U(0, 1)$.

Independence. The probability of the next value falling in a given interval does not depend on the values already drawn – successive R_i must be statistically uncorrelated, so that knowing R_i gives no information about R_{i+1} . A plot of R_{i+1} against R_i for an independent sequence should look like a featureless scatter of points filling the unit square, with no visible line, curve or clustering.

Both properties are formally checked later, in Section 6.4: uniformity by the Kolmogorov–Smirnov or chi-square goodness-of-fit tests, independence by the autocorrelation test. Section 6.1 only states *what* is required; Section 6.4 states *how* it is verified.

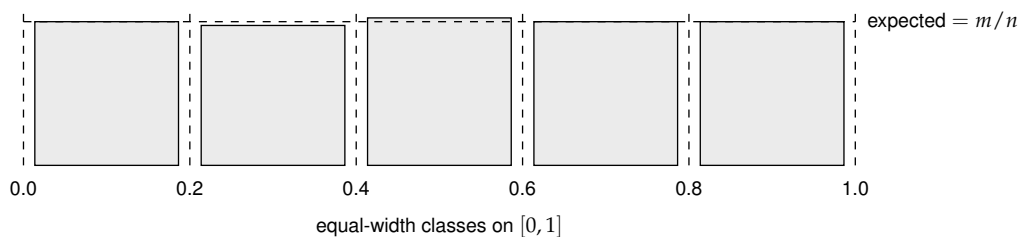


Figure 17: Uniformity, illustrated: if $[0, 1]$ is split into $n = 5$ equal classes and m numbers are generated, each class should receive about the same count, m/n – a good generator gives bars of roughly equal height.

If the underlying generator is defective, the produced sequence departs from this ideal in recognisable ways:

Departure	What is observed
Non-uniformity	Some sub-intervals of $[0, 1]$ are favoured over others; the sample mean or variance drifts away from the theoretical 0.5 and $1/12$.
Autocorrelation	Successive numbers are correlated instead of independent, so cyclic patterns appear – e.g. a run of values continuously above the mean followed by a run continuously below it.
Gaps and patterns	Some values or regions of $[0, 1]$ never appear, or values recur in a visible, exploitable pattern.
Short period	The sequence starts repeating itself after relatively few numbers, so the “randomness” runs out well before the simulation does.
Discreteness	The generator effectively produces only a limited set of discrete values instead of (numerically) covering the continuum on $[0, 1]$.

Because of this, a generator must always be statistically tested (Section 6.4) before it is trusted for a simulation study – a generator can look plausible on a handful of numbers and still fail badly over the millions of draws a real simulation consumes. A single moderately sized discrete-event simulation (for example, a queueing model run for a few thousand simulated customers, each needing an interarrival time and a service time) can easily consume tens of thousands of random numbers in one run, and a full study repeats such runs many times over with different seeds; even a small, systematic departure from uniformity or independence therefore has ample opportunity to bias the results.

6.2 Generation of Pseudo-Random Numbers

Pseudo-random numbers

Pseudo means false. A pseudo-random number is produced by a known, deterministic arithmetic recurrence rather than by a genuinely random physical process; because the generating rule is known, the same seed always reproduces exactly the same sequence, so the numbers are not *truly* random. In practice, however, a well-designed generator produces sequences that statistically behave enough like true random numbers – satisfying uniformity and independence – to be used safely in simulation.

Reproducibility is in fact a deliberate advantage for simulation, not a defect: starting from the same seed, an experiment can be replicated exactly, which is essential for **debugging** a model (a fault can be reproduced and stepped through) and for comparing two competing system designs under *identical* random conditions – if both designs are fed the same underlying stream of R_i values, any difference in output is attributable to the design change itself and not to random noise, which sharpens the comparison considerably.

Random numbers used to drive a simulation can be obtained in three general ways:

1. **Tables.** Reading from a pre-computed table of random digits stored in the computer’s memory (e.g. the RAND Corporation’s published tables). This guarantees good statistical properties, since the table can be exhaustively tested once, but it is limited in length, slow to access for very large runs, and needs storage proportional to how many numbers are needed.
2. **Electronic devices.** A dedicated hardware generator built from a physically random process (thermal or electronic noise, radioactive decay). These are genuinely, not merely pseudo, random, but the equipment is expensive, the stream cannot be exactly reproduced for debugging, and access is comparatively slow.
3. **Arithmetic operation.** Computing each number from the previous one(s) by a fixed formula, evaluated entirely in software.

Arithmetic generation is by far the most widely used method in practice, since it is fast, needs almost no memory (only the current state), is exactly reproducible from a seed, and – when the recurrence and

its parameters are chosen well – passes the statistical tests of Section 6.4 just as convincingly as a table or hardware source. The linear and multiplicative congruential methods of Section 6.3 are the standard arithmetic generators used in simulation software.

Method	Speed	Reproducible?	Memory / cost
Random-number table	Slow for large runs	Yes, if the table is stored	Storage grows with the quantity needed
Electronic device	Fast, but access is comparatively slow	No – output cannot be replicated exactly	Expensive dedicated hardware
Arithmetic operation	Fast (a few arithmetic operations per number)	Yes, exactly, from the seed	Only the current state is held; negligible

A good random-number generation procedure should: be **fast**; be **portable** to different computers/languages; have a **sufficiently long period** before the sequence repeats; be **replicable** from a given seed (so an experiment can be reproduced exactly); hold the ideal statistical properties of **uniformity** and **independence**; and require **little computer memory**. Equivalently: uniform, independent, reproducible, non-repeating for the length required, fast, and economical.

6.3 Linear and Multiplicative Congruential Methods

Congruential generator

The linear congruential method generates a sequence of integers $X_0, X_1, X_2, \dots$ between 0 and $m - 1$ by the recursive relation

$$X_{i+1} = (aX_i + c) \bmod m, \quad i = 0, 1, 2, \dots$$

and converts each X_i into a random number on $[0, 1)$ by

$$R_i = \frac{X_i}{m}.$$

X_0 is the **seed**, a the **multiplier**, c the **increment** and m the **modulus**. Since X_i can take only the m integer values $0, 1, \dots, m - 1$, the sequence must eventually repeat, with period at most m .

The seed must be chosen with some care. In the multiplicative case ($c = 0$), a seed of $X_0 = 0$ is fatal: every subsequent $X_i = (a \cdot 0) \bmod m = 0$, so the generator collapses to the constant sequence $0, 0, 0, \dots$ – this is why the maximum-period condition for the multiplicative method explicitly requires X_0 to be **odd** (and hence non-zero). In the mixed case ($c \neq 0$) any seed in $\{0, 1, \dots, m - 1\}$ is admissible, since the additive term c prevents the sequence from ever locking onto 0. On real hardware, computing aX_i can also overflow the machine's integer word before the $\bmod m$ is applied; production implementations avoid this either by working in a wider integer type or by a technique such as Schrage's method, which restructures the multiplication and modulus so that no intermediate value exceeds m .

The method was introduced by D. H. Lehmer in 1949 and remains, in one variant or another, the most widely taught random-number generator because the recurrence is trivial to implement, cheap to evaluate, and – unlike a black-box hardware source – completely transparent: its period and its statistical behaviour can be analysed exactly, which is precisely what the rest of this section does.

Two named special cases of the same recurrence:

- **Mixed (linear) congruential method:** $c \neq 0$ – both the multiplicative term aX_i and the additive term c are present.
- **Multiplicative congruential method:** $c = 0$, so $X_{i+1} = (aX_i) \bmod m$. The modulus is usually chosen to match the computer's word size: $m = 2^b$ on a binary machine (b = bits per word) or $m = 10^d$ on a

decimal machine (d = digits per word).

	Mixed ($c \neq 0$)	Multiplicative ($c = 0$)
Arithmetic per step	one multiply, one add, one mod	one multiply, one mod (cheaper)
Best achievable period	full period $P = m$	at most $P = m/4$ for $m = 2^k$ (or $P = m - 1$ for m prime)
Typical use	general purpose, any m	the more common choice in practice, because of its speed

Procedure, step by step.

1. Choose the modulus m (usually 2^b or 10^d , matching the word size), the multiplier a , the increment c (zero for the multiplicative method), and a seed X_0 .
2. Check the relevant maximum-period condition for the chosen case, so the period is known in advance to be adequate for the run length planned.
3. Iterate $X_{i+1} = (aX_i + c) \bmod m$ for $i = 0, 1, 2, \dots$, keeping only the current X_i in memory.
4. After each new X_i , output $R_i = X_i/m$ as the next $(0, 1)$ random number consumed by the simulation.
5. Stop after the required number of draws, or earlier if X_i returns to X_0 (the sequence has completed one full cycle and would only repeat from here on).

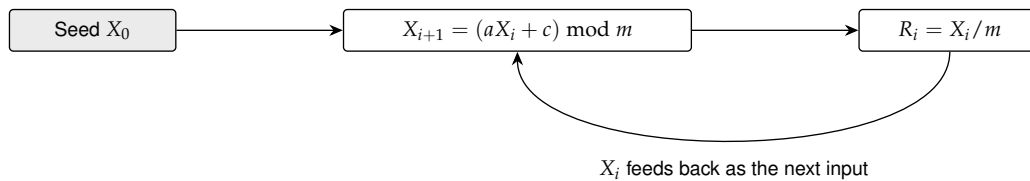


Figure 18: The congruential recurrence: each X_i both yields R_i and feeds back into the recurrence to produce X_{i+1} .

Maximum-period conditions

Let P denote the **period** of the generator – the number of distinct values produced before the sequence repeats itself. The theoretical maximum is $P = m$.

Mixed congruential ($c \neq 0$). The full period $P = m$ is obtained if and only if:

1. c and m are relatively prime, $\gcd(c, m) = 1$;
2. $a - 1$ is divisible by every prime factor of m ;
3. $a - 1$ is divisible by 4 if m is divisible by 4.

For the common choice $m = 2^b$, the only prime factor of m is 2, so conditions (2)-(3) collapse to: $a - 1$ divisible by 4, i.e. $a = 1 + 4k$ for some integer k ; together with condition (1) this gives the maximum period $P = m = 2^b$.

Multiplicative congruential ($c = 0$). With $m = 2^k$, condition (1) can never hold (since $c = 0$ is never coprime to m once $m > 1$), so the full period m is unreachable; the best obtainable period is only

$$P = \frac{m}{4} = 2^{k-2},$$

achieved when the seed X_0 is **odd** and the multiplier satisfies $a \equiv 3 \pmod{8}$ or $a \equiv 5 \pmod{8}$, i.e. $a = 3 + 8k$ or $a = 5 + 8k$ for some integer $k \geq 0$.

Multiplicative congruential, m prime. If instead m is chosen to be a prime number (with $c = 0$), the maximum achievable period is

$$P = m - 1,$$

obtained when the multiplier a is a *primitive root* of m – that is, the smallest positive integer k for which $a^k - 1$ is exactly divisible by m is $k = m - 1$ itself (no smaller power of a returns to 1 modulo m).

Whichever case applies, satisfying the period condition is necessary but not sufficient: a generator can achieve the full period allowed by its case and still fail the statistical tests of Section 6.4. The classic cautionary example is **RANDU** ($a = 65539$, $c = 0$, $m = 2^{31}$), once standard on IBM mainframes: it satisfies the period condition above, yet successive triples (R_i, R_{i+1}, R_{i+2}) were later shown to fall on only a handful of parallel planes when plotted in three dimensions, badly violating independence. This is why the multiplier and modulus of a production generator (e.g. the Lewis–Goodman–Miller “minimal standard” generator, $a = 16807$, $m = 2^{31} - 1$, a prime) are chosen only after extensive testing, not from the period condition alone.

Common mistakes to avoid. Always compute $R_i = X_i/m$, never X_{i+1}/m – the subscripts must match. The period can never exceed m , so a computed period larger than m signals an arithmetic slip. For the mixed method, check *all three* Hull–Dobell-type conditions before claiming the full period $P = m$; satisfying only one or two is not enough. For the multiplicative method with $m = 2^k$, remember X_0 must be **odd** – an even seed (and, worse, $X_0 = 0$) breaks the generator – and the best possible period is only $m/4$, not m . Finally, a generator meeting its maximum-period condition is not automatically “random enough” for a simulation: the period condition only rules out premature cycling, it says nothing about independence, so the sequence must still pass the tests of Section 6.4.

Example: Checking the maximum-period conditions and generating the sequence (mixed congruential).

Let $a = 5$, $c = 43$, $m = 4$, $X_0 = 1$.

Step 1 – check the conditions. $m = 2^2 = 4$ is a power of 2.

- $\gcd(c, m) = \gcd(43, 4)$: $43 = 4(10) + 3 \Rightarrow \gcd(43, 4) = \gcd(4, 3) = \gcd(3, 1) = 1$. Condition (1) holds.
- The only prime factor of $m = 4$ is 2, and $a - 1 = 5 - 1 = 4$ is divisible by 2. Condition (2) holds.
- $m = 4$ is divisible by 4, and $a - 1 = 4$ is divisible by 4 (take $k = 1$: $5 = 1 + 4(1)$). Condition (3) holds.

All three conditions hold, so the maximum period $P = m = 4$ is guaranteed.

Step 2 – generate the sequence and confirm $P = 4$. $X_{i+1} = (5X_i + 43) \bmod 4$, $R_i = X_i/4$.

$$\begin{array}{ll}
 X_1 = (5 \cdot 1 + 43) \bmod 4 = 48 \bmod 4 = 0, & R_1 = 0/4 = 0 \\
 X_2 = (5 \cdot 0 + 43) \bmod 4 = 43 \bmod 4 = 3, & R_2 = 3/4 = 0.75 \\
 X_3 = (5 \cdot 3 + 43) \bmod 4 = 58 \bmod 4 = 2, & R_3 = 2/4 = 0.5 \\
 X_4 = (5 \cdot 2 + 43) \bmod 4 = 53 \bmod 4 = 1, & R_4 = 1/4 = 0.25 \\
 X_5 = (5 \cdot 1 + 43) \bmod 4 = 48 \bmod 4 = 0, & R_5 = 0/4 = 0
 \end{array}$$

i	1	2	3	4	5
X_i	0	3	2	1	0
R_i	0	0.75	0.5	0.25	0

Since $X_5 = X_1 = 0$, the sequence repeats after exactly 4 steps, confirming empirically that $P = m = 4$, exactly as predicted.

Example: Generating a sequence with the mixed congruential method.

Let $X_0 = 27$, $a = 17$, $c = 43$, $m = 100$; generate 5 values. $X_{i+1} = (17X_i + 43) \bmod 100$, $R_i = X_i/100$.

$$\begin{aligned} X_1 &= (17 \cdot 27 + 43) \bmod 100 = 502 \bmod 100 = 2, & R_1 &= 0.02 \\ X_2 &= (17 \cdot 2 + 43) \bmod 100 = 77 \bmod 100 = 77, & R_2 &= 0.77 \\ X_3 &= (17 \cdot 77 + 43) \bmod 100 = 1352 \bmod 100 = 52, & R_3 &= 0.52 \\ X_4 &= (17 \cdot 52 + 43) \bmod 100 = 927 \bmod 100 = 27, & R_4 &= 0.27 \\ X_5 &= (17 \cdot 27 + 43) \bmod 100 = 502 \bmod 100 = 2, & R_5 &= 0.02 \end{aligned}$$

i	1	2	3	4	5
X_i	2	77	52	27	2
R_i	0.02	0.77	0.52	0.27	0.02

The generated random numbers are 0.02, 0.77, 0.52, 0.27, 0.02, ... Since $X_5 = X_1$, this seed already cycles with period 4 – far short of the theoretical maximum $m = 100$. This is consistent with the conditions above: $\gcd(43, 100) = 1$ holds, but $m = 100 = 2^2 \cdot 5^2$ has prime factor 5, and $a - 1 = 16$ is *not* divisible by 5, so condition (2) fails and the full period is lost.

Example: Generating a sequence with the multiplicative congruential method.

Let $X_0 = 117$, $a = 43$, $m = 1000$ (so $c = 0$); generate 3 values. $X_{i+1} = (43X_i) \bmod 1000$, $R_i = X_i/1000$.

$$\begin{aligned} X_1 &= (43 \cdot 117) \bmod 1000 = 5031 \bmod 1000 = 31, & R_1 &= 31/1000 = 0.031 \\ X_2 &= (43 \cdot 31) \bmod 1000 = 1333 \bmod 1000 = 333, & R_2 &= 333/1000 = 0.333 \\ X_3 &= (43 \cdot 333) \bmod 1000 = 14319 \bmod 1000 = 319, & R_3 &= 319/1000 = 0.319 \end{aligned}$$

i	1	2	3
X_i	31	333	319
R_i	0.031	0.333	0.319

The generated random numbers are 0.031, 0.333, 0.319. Note that only one multiply-and-mod is needed per step (no addition), which is exactly the computational saving that makes the multiplicative method the more popular choice in production random-number libraries, provided a and m are chosen carefully enough to give both a long period and good independence.

Together the three worked examples cover the whole of Section 6.3 end to end: the first shows *how to decide, before generating anything*, whether a proposed (a, c, m) will give the longest possible period; the second and third show *how to generate* the X_i and R_i sequence itself, once for each named variant of the recurrence. Any exam question on this topic reduces to one of these two tasks – checking a period condition, or iterating the recurrence a fixed number of times – so it is worth being completely fluent with the arithmetic of both before moving on to how the resulting sequence is tested for randomness in Section 6.4.

Example	a, c, m	X_0	Period found	Purpose
Max. period check	5, 43, 4	1	$P = 4 = m$ (full)	verify Hull–Dobell conditions
Mixed sequence	17, 43, 100	27	4 (partial, $\ll m$)	generate R_i , mixed method
Multiplicative	43, 0, 1000	117	not evaluated	generate R_i , multiplicative method

Requirements for a good pseudo-random generator, summarised. (1) The generated sequence must follow the uniform $(0, 1)$ distribution; (2) it must be statistically independent; (3) it must be reproducible, so a simulation experiment can be replicated exactly; (4) it must be non-repeating for whatever length is required – a long period is essential; (5) generation must be fast, since simulation studies consume very large quantities of random numbers; (6) the technique should require little computer memory.

6.4 Tests for Random Numbers

A pseudo-random generator is only useful if the numbers it outputs behave like genuine random observations. Two properties are checked statistically: **uniformity** (do the R_i look like a sample from $U(0, 1)$?) and **independence** (are successive R_i statistically unrelated?). Each test states a null hypothesis and is run at a chosen significance level α .

Uniformity: $H_0 : R_i \sim U(0, 1)$ vs. $H_1 : R_i \not\sim U(0, 1)$ — tested by the **K–S test** and the **chi-square test**.

Independence: $H_0 : R_i$ independently generated vs. $H_1 : R_i$ not independent — tested by the **gap test**, **poker test** and **autocorrelation test**.

$\alpha = P(\text{reject } H_0 \mid H_0 \text{ true})$. Failing to reject H_0 only means the test found no evidence against that property; it does not prove it.

Kolmogorov–Smirnov (K–S) Test

K–S test statistic

Compares the empirical CDF of the sorted sample to the theoretical $U(0, 1)$ CDF $F(x) = x$. Rank the data ascending, $R_{(1)} \leq R_{(2)} \leq \dots \leq R_{(N)}$, and compute

$$D^+ = \max_i \left\{ \frac{i}{N} - R_{(i)} \right\}, \quad D^- = \max_i \left\{ R_{(i)} - \frac{i-1}{N} \right\}, \quad D = \max(D^+, D^-).$$

D is compared against the tabulated critical value $D_{\alpha, N}$.

Example: the five numbers 0.44, 0.81, 0.14, 0.05, 0.93 were generated; test for uniformity at $\alpha = 0.05$.

$H_0 : R_i \sim U(0, 1)$; $H_1 : R_i \not\sim U(0, 1)$. Sort ascending and tabulate, $N = 5$:

i	1	2	3	4	5
$R_{(i)}$	0.05	0.14	0.44	0.81	0.93
i/N	0.20	0.40	0.60	0.80	1.00
$i/N - R_{(i)}$	0.15	0.26	0.16	−0.01	0.07
$R_{(i)} - (i-1)/N$	0.05	−0.06	0.04	0.21	0.13

$D^+ = \max\{0.15, 0.26, 0.16, -0.01, 0.07\} = 0.26$ (at $i = 2$). $D^- = \max\{0.05, -0.06, 0.04, 0.21, 0.13\} = 0.21$ (at $i = 4$). So $D = \max(0.26, 0.21) = 0.26$.

From the K–S table, $D_{0.05, 5} = 0.56327$. Since $D = 0.26 < 0.56327$, H_0 is not rejected: no evidence of non-uniformity is detected.

Decision rule: if $D > D_{\alpha, N}$, reject H_0 (not uniform); if $D \leq D_{\alpha, N}$, do not reject H_0 .

Chi-Square Test

Chi-square test statistic

Bin the N observations into n equal class intervals over $[0, 1)$ (or the raw range for integer data). With O_i the observed and $E_i = N/n$ the expected count per class,

$$\chi_0^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i},$$

which is approximately chi-square distributed with $n - 1$ degrees of freedom. Better suited than K-S for large N .

Example: a multiplicative congruential generator produced 100 two-digit numbers (00–99). Binning into $n = 10$ class intervals of width 10 gives the observed counts below. Test uniformity at the 95% confidence level.

$H_0 : R_i \sim U(0, 1)$; $H_1 : R_i \not\sim U(0, 1)$. Expected count per class: $E_i = N/n = 100/10 = 10$.

Class interval	O_i	$O_i - E_i$	$(O_i - E_i)^2$	$(O_i - E_i)^2/E_i$
$0 < r \leq 10$	13	3	9	0.9
$10 < r \leq 20$	7	-3	9	0.9
$20 < r \leq 30$	12	2	4	0.4
$30 < r \leq 40$	13	3	9	0.9
$40 < r \leq 50$	7	-3	9	0.9
$50 < r \leq 60$	13	3	9	0.9
$60 < r \leq 70$	5	-5	25	2.5
$70 < r \leq 80$	10	0	0	0.0
$80 < r \leq 90$	12	2	4	0.4
$90 < r \leq 100$	8	-2	4	0.4
Total	100	-	82	8.2

Check: $\sum O_i = 13 + 7 + 12 + 13 + 7 + 13 + 5 + 10 + 12 + 8 = 100 = N$, as required. $\chi_0^2 = 82/10 = 8.2$. Degrees of freedom = $n - 1 = 9$. From the chi-square table, $\chi_{0.05,9}^2 = 16.919$. Since $\chi_0^2 = 8.2 < 16.919$, H_0 is not rejected: the numbers are acceptable as uniform.

Decision rule: if $\chi_0^2 > \chi_{\alpha, n-1}^2$, reject H_0 (not uniform); otherwise do not reject.

Gap Test

Gap test

A **gap** of length x is the number of digits between two successive occurrences of a specified digit d . Since each digit has probability 0.1 of occurring, the theoretical cumulative distribution of gap length is geometric:

$$F(x) = P(\text{gap} \leq x) = 1 - (0.9)^{x+1}.$$

The observed gaps are grouped into the same class intervals and compared to $F(x)$ with the K-S statistic $D = \max |F(x) - S_N(x)|$, where $S_N(x)$ is the observed cumulative relative frequency; D is then checked against the K-S critical value D_α (independence, not uniformity, is being tested here).

Example: a sequence of 110 digits is analysed for the gaps between repetitions of each digit 0–9, at $\alpha = 0.05$.

$H_0 : R_i$ independently generated; $H_1 : R_i$ not independent. Number of gaps = number of digits – number of distinct digits = $110 - 10 = 100$.

Gap lengths observed for each digit, and the count of gaps they contribute:

Digit	Gap lengths	No. of gaps
0	47, 9, 3, 2, 2, 21, 6	7
1	2, 7, 6, 10, 6, 45, 9, 10	8
2	1, 6, 12, 22, 0, 38, 8, 4	8
3	10, 7, 0, 2, 3, 8, 5, 5, 2, 4, 1, 14, 0, 9, 14, 5, 1	17
4	17, 7, 5, 0, 2, 18, 12, 3, 23, 0	10
5	10, 26, 0, 3, 2, 11, 2, 0, 0, 12, 2, 12, 2	13
6	3, 9, 24, 9, 14, 0, 22	7
7	4, 5, 26, 10, 1, 16, 6, 22	8
8	19, 7, 3, 8, 21, 16, 0, 0, 4	9
9	5, 5, 7, 8, 4, 1, 14, 0, 14, 2, 8, 1, 9	13

Total gaps = 7 + 8 + 8 + 17 + 10 + 13 + 7 + 8 + 9 + 13 = 100, matching the count above.

All 100 gap lengths are grouped into classes of width 4 and compared to $F(x) = 1 - (0.9)^{x+1}$ evaluated at the upper class boundary:

Gap length	Freq.	$S_N(x)$	Cum. $S_N(x)$	$F(x)$	$ F(x) - S_N(x) $
0–3	35	0.35	0.35	0.3439	0.0061
4–7	22	0.22	0.57	0.5695	0.0005
8–11	17	0.17	0.74	0.7176	0.0224
12–15	9	0.09	0.83	0.8147	0.0153
16–19	5	0.05	0.88	0.8784	0.0016
20–23	6	0.06	0.94	0.9202	0.0198
24–27	3	0.03	0.97	0.9497	0.0223
28–31	0	0.00	0.97	0.9657	0.0043
32–35	0	0.00	0.97	0.9775	0.0075
36–39	2	0.02	0.99	0.9852	0.0043
40–43	0	0.00	0.99	0.9903	0.0003
44–47	1	0.01	1.00	0.9936	0.0064

Check: frequencies sum to 35 + 22 + 17 + 9 + 5 + 6 + 3 + 0 + 0 + 2 + 0 + 1 = 100. The largest deviation is $D = \max |F(x) - S_N(x)| = 0.0224$, at the class 8–11.

Critical value (asymptotic K–S formula for $\alpha = 0.05$): $D_{0.05} = 1.36/\sqrt{N} = 1.36/\sqrt{100} = 0.136$.

Since $D = 0.0224 < D_{0.05} = 0.136$, H_0 is not rejected: no evidence of dependence is detected on the basis of this test.

Poker's Method

Poker test

Digits are grouped into k -digit numbers, treated like a poker hand, and classified by repeated-digit pattern. The probability of each pattern is computed from simple digit-matching arguments, $E_i = NP_i$, and the observed pattern counts O_i are compared to E_i with the usual chi-square statistic, $\chi_0^2 = \sum (O_i - E_i)^2 / E_i$, on $n - 1$ degrees of freedom (n = number of patterns). The same method applies to 4-digit and 5-digit groupings (four and seven possible patterns respectively), with the pattern probabilities recomputed from the longer digit string; only the 3-digit case is worked in full below.

For 3-digit numbers there are three possible patterns: all different, exactly one pair, or all three alike.

$$P(3 \text{ different}) = (0.9)(0.8) = 0.72, \quad P(3 \text{ alike}) = (0.1)(0.1) = 0.01, \quad P(\text{one pair}) = 1 - 0.72 - 0.01 = 0.27.$$

Example: a sequence of 1000 three-digit numbers is generated: 680 have three different digits, 289 contain exactly one pair, and 31 contain three like digits. Test independence at $\alpha = 0.05$.

H_0 : R_i independently generated; H_1 : not independent. $E_i = P_i \times 1000$: $E_1 = 720$, $E_2 = 10$, $E_3 = 270$.

Combination	O_i	E_i	$O_i - E_i$	$(O_i - E_i)^2 / E_i$
Three different digits	680	720	-40	2.22
Exactly one pair	289	270	19	1.34
Three like digits	31	10	21	44.10
Total	1000	1000	-	47.66

Check: E_i total = $720 + 10 + 270 = 1000 = N$; O_i total = $680 + 289 + 31 = 1000$. $\chi_0^2 = 2.22 + 1.34 + 44.10 = 47.66$.

Degrees of freedom = $n - 1 = 3 - 1 = 2$; from the chi-square table, $\chi_{0.05,2}^2 = 5.99$. Since $\chi_0^2 = 47.66 > 5.99$, H_0 is **rejected**: independence of the numbers is not supported by this test.

Decision rule (poker and chi-square): reject H_0 if $\chi_0^2 > \chi_{\alpha, n-1}^2$; otherwise do not reject.

Autocorrelation Test

Autocorrelation test

Tests whether numbers m apart in the sequence (lag m , starting index i) are correlated. With N the sample size, find the largest integer M with $i + (M + 1)m \leq N$, then

$$\hat{\rho}_{im} = \left[\frac{1}{M+1} \sum_{k=0}^M R_{i+km} R_{i+(k+1)m} \right] - 0.25, \quad \sigma_{\hat{\rho}} = \frac{\sqrt{13M+7}}{12(M+1)}, \quad Z_0 = \frac{\hat{\rho}_{im}}{\sigma_{\hat{\rho}}}.$$

Example: a sequence of 30 numbers $R_1, \dots, R_{30}$ is generated:

$i = 1..10$	0.12	0.01	0.23	0.28	0.89	0.31	0.64	0.28	0.83	0.93
$i = 11..20$	0.99	0.15	0.33	0.35	0.91	0.41	0.60	0.27	0.75	0.88
$i = 21..30$	0.68	0.49	0.05	0.43	0.95	0.58	0.19	0.36	0.69	0.87

Test whether the numbers 5 apart, starting at the 3rd, are autocorrelated ($\alpha = 0.05$, $z_{\alpha/2} = 1.96$).

$H_0 : \rho_{im} = 0$; $H_1 : \rho_{im} \neq 0$. Here $i = 3$, $m = 5$, $N = 30$.

Largest M : $3 + (M + 1)(5) \leq 30 \Rightarrow M + 1 \leq 5.4 \Rightarrow M \leq 4.4$, so $M = 4$ (largest integer).

Indices used: $R_3, R_8, R_{13}, R_{18}, R_{23}, R_{28} = 0.23, 0.28, 0.33, 0.27, 0.05, 0.36$.

k	0	1	2	3	4
$R_{3+5k} \cdot R_{3+5(k+1)}$	0.23×0.28	0.28×0.33	0.33×0.27	0.27×0.05	0.05×0.36
value	0.0644	0.0924	0.0891	0.0135	0.0180

Sum = $0.0644 + 0.0924 + 0.0891 + 0.0135 + 0.0180 = 0.2774$.

$$\hat{\rho}_{3,5} = \frac{0.2774}{5} - 0.25 = 0.05548 - 0.25 = -0.19452.$$

$$\sigma_{\hat{\rho}} = \frac{\sqrt{13(4)+7}}{12(5)} = \frac{\sqrt{59}}{60} = \frac{7.6811}{60} = 0.1280.$$

$$Z_0 = \frac{-0.19452}{0.1280} = -1.52.$$

Since $|Z_0| = 1.52 < z_{0.025} = 1.96$, H_0 is not rejected: no evidence of autocorrelation at lag 5 starting from index 3.

Decision rule: reject H_0 if $|Z_0| > z_{\alpha/2}$ (autocorrelation present); otherwise do not reject.

6.5 Generating Discrete Distributions

Real applications usually need non-uniform random variates: a different probability attaches to each possible outcome. The general recipe reuses the uniform generator: build the cumulative distribution of the target discrete variable, partition $[0, 1)$ into intervals sized to match each outcome's probability, draw $U \sim U(0, 1)$, and read off the outcome whose interval contains U .

Discrete-variate generation rule

For a discrete variable X taking values $x_1 < x_2 < \dots < x_k$ with cumulative probabilities $y_0 = 0, y_1, \dots, y_k = 1$, draw $U \sim U(0, 1)$ and set $X = x_j$ where j is the smallest index with $y_{j-1} < U \leq y_j$.

Example: the number of items bought by a shopper at a store is recorded for 250 customers.

X (items)	N (customers)	$P(X) = N/250$	Cumulative $P(X)$	Interval for U
1	25	0.10	0.10	$(0.00, 0.10]$
2	128	0.51	0.61	$(0.10, 0.61]$
3	47	0.19	0.80	$(0.61, 0.80]$
4	38	0.15	0.95	$(0.80, 0.95]$
5	12	0.05	1.00	$(0.95, 1.00]$

Check: N totals $25 + 128 + 47 + 38 + 12 = 250$; $P(X)$ totals $0.10 + 0.51 + 0.19 + 0.15 + 0.05 = 1.00$; the cumulative column runs from 0 to 1, so the five intervals partition $[0, 1)$ exactly.

To generate an outcome, draw U and locate its interval. For $U = 0.05$: since $0.00 < 0.05 \leq 0.10$, $X = 1$. For $U = 0.72$: since $0.61 < 0.72 \leq 0.80$, $X = 3$. For $U = 0.97$: since $0.95 < 0.97 \leq 1.00$, $X = 5$.

A computer routine searches the interval table from the top, so it is fastest to list intervals in *decreasing order of probability* (here: $X = 2$ first, width 0.51, then $X = 3, 4, 1, 5$): most draws are then resolved after checking only the first one or two rows, without changing which X any given U maps to.

6.6 Inversion, Rejection, Composition and Convolution Methods

Four general-purpose techniques generate variates from an arbitrary continuous distribution once a uniform generator is available.

Inversion Method

Inverse-transform method

If $U \sim U(0, 1)$ and F^{-1} is the inverse of the target CDF, then $X = F^{-1}(U)$ has the target distribution F . Steps: (1) derive $F(x)$; (2) set $F(x) = R$ for a uniform draw R ; (3) solve algebraically for x in terms of R ; (4) substitute successive R values to generate variates.

Example: derive the generator for the exponential distribution with rate λ , $F(x) = 1 - e^{-\lambda x}$, $x \geq 0$.

Set $F(x) = R$:

$$1 - e^{-\lambda x} = R \implies e^{-\lambda x} = 1 - R \implies -\lambda x = \ln(1 - R) \implies x = -\frac{1}{\lambda} \ln(1 - R).$$

Since $R \sim U(0, 1)$ implies $1 - R \sim U(0, 1)$ as well, $\ln(1 - R)$ and $\ln(R)$ have the same distribution, so the equivalent form $x = -\frac{1}{\lambda} \ln(R)$ is also used — the two give the same distribution of x , though not the same value for one specific draw of R .

Numeric check: $\lambda = 0.5$, drawn $R = 0.75$: $1 - R = 0.25$, $\ln(0.25) = -1.3863$, so

$$x = -\frac{1}{0.5} \times (-1.3863) = 2 \times 1.3863 = 2.7726.$$

Rejection (Acceptance–Rejection) Method

Rejection algorithm

Applies when the target density $f(x)$ is bounded on a finite range $[a, b]$ by some constant $c \geq \max_{x \in [a, b]} f(x)$.

1. Generate two independent uniform variates $U_1, U_2 \sim U(0, 1)$.
2. Compute the candidate point $X_0 = a + U_1(b - a)$.
3. Compute the height $Y_0 = cU_2$.
4. If $Y_0 \leq f(X_0)$, accept X_0 as the output; otherwise discard both variates and repeat from step 1.

Geometrically, $f(x)$ is enclosed in a rectangle of width $b - a$ and height c ; (X_0, Y_0) is a uniformly random point in that rectangle, and it is kept only when it falls under the curve.

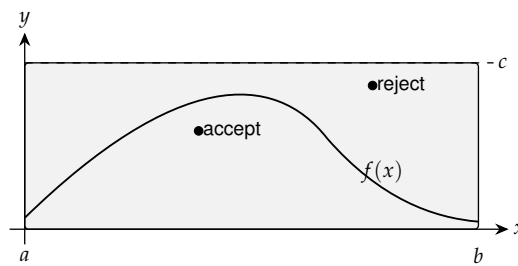


Figure 19: Rejection method: accept a point under $f(x)$, reject a point above it.

Composition Method

Composition algorithm

Used when the target density can be written as a probability-weighted mixture of simpler densities, $f(x) = \sum_j p_j f_j(x)$ with $\sum_j p_j = 1$.

1. Generate a uniform variate $U \sim U(0, 1)$ and use it to select an index J with $P(J = j) = p_j$ (e.g. by the discrete-variate rule of Section 6.5).
2. Generate a fresh variate X from the chosen component density f_J , using whichever method suits f_J (inversion, rejection, ...).
3. Return X as the desired output.

This is useful when $f(x)$ itself has no convenient closed-form inverse but each f_j does.

Convolution Method

Convolution algorithm

Used when the target variable is a **sum** of n independent random variables, $X = Y_1 + Y_2 + \dots + Y_n$ (the distribution of a sum is called the convolution of the individual distributions).

1. Generate n independent variates $Y_1, \dots, Y_n$, one from each component distribution.
2. Sum them: $X = Y_1 + Y_2 + \dots + Y_n$.
3. Return X as the desired output.

Standard special cases: the sum of n independent Bernoulli variates is binomial; the sum of n independent exponential variates (same rate λ) is n -Erlang. The same summing idea, via the central limit theorem, generates approximately normal variates: for i.i.d. $Y_1, \dots, Y_n$ with mean μ and variance

σ^2 ,

$$Z = \frac{\sum_{i=1}^n Y_i - n\mu}{\sigma\sqrt{n}} = \frac{\sqrt{n}(\bar{Y}_n - \mu)}{\sigma}$$

is approximately standard normal for large n , so $\mu + \sigma Z$ (with Z built from a sum of uniforms) gives an approximately $N(\mu, \sigma^2)$ variate.

Chapter 7 Verification and Validation of Simulation Models

7.1 Verification and Validation

Before a simulation model's results can be trusted, the model must be shown both to be technically correct and to be a faithful stand-in for the system it represents. These are two distinct checks.

Verification vs. validation

Verification is the process of confirming that a model is correctly implemented with respect to the conceptual model – that its input parameters and logical structure are correctly represented in the computer program. It is largely a software-engineering activity (debugging the code against the specification) and answers: *“Have we built the model right?”*

Validation is the determination that a model is an accurate representation of the real system. It is usually achieved through **calibration**: an iterative process of comparing the model's behaviour with the real system's behaviour and using the discrepancies to improve the model, repeated until the model's accuracy is judged acceptable. It answers: *“Have we built the right model?”* Validation compares the computational model against either measurements of the actual system or an analytical (mathematical) model of it, and is, by necessity, always an incomplete activity.

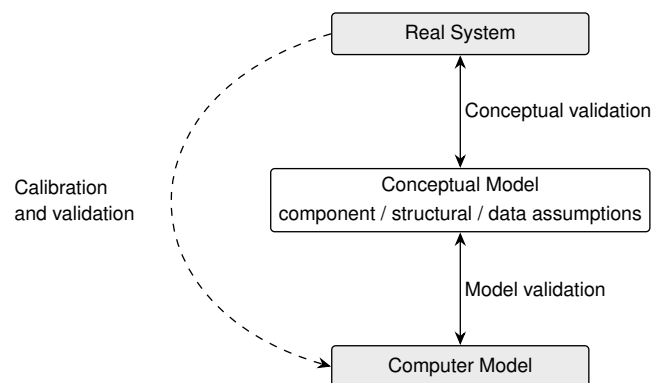


Figure 20: Real system, conceptual model and computer model: verification checks the Conceptual Model → Computer Model translation; validation checks each stage against the Real System.

Verification = build the model right (internal correctness, code vs. specification). **Validation** = build the right model (external accuracy, model vs. reality), achieved by iterative calibration.

7.2 Verification of Simulation Models

Verification makes sure the conceptual model is translated into a computerised model properly and truthfully. The following techniques are used:

- **Independent review:** have the computerised model checked by someone other than its developer.
- **Flow-diagram tracing:** draw a flow diagram covering every logically possible action the system can take at each event, and follow the model logic for each action and event type.
- **Output examination:** closely examine model output under a variety of input-parameter settings for reasonableness.
- **Interactive debugging:** a debugger lets the analyst or programmer step through and test the simulation program interactively; it is an essential component of building a correct model.
- **Comparison with known results:** for a system with known theoretical behaviour (e.g. a simple queue), compare simulation output against the analytical result as an upper or lower bound.
- **Documentation review:** describing in detail what the programmer implemented, in design and code, often exposes problems by itself.

- **Trace:** manually step through a few iterations of the program and check that the behaviour is reasonable.

7.3 Calibration and Validation of Models

Validation is the overall process of comparing the model and its behaviour with the real system and its behaviour. **Calibration** is the iterative process of comparing the model to the real system, adjusting the model, comparing the revised model to reality again, and so on, until the model is accepted (validated). The comparison of model to reality is carried out by two kinds of test: a **subjective test** – talking to people knowledgeable about the system and forming a judgement – and an **objective test** – one or more statistical tests comparing model output with the assumptions built into the model.

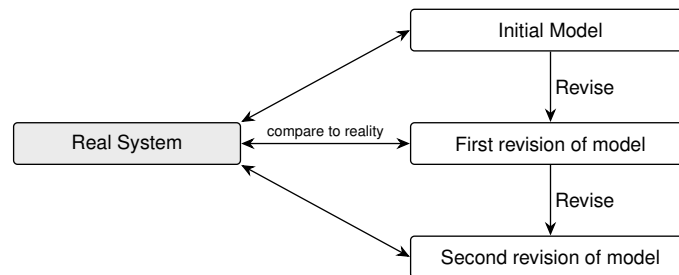


Figure 21: Iterative process of calibrating a model: build, compare to reality, revise, compare again.

This iterative loop breaks down into three parts (these are the same three parts elaborated for the Naylor and Finger process, §7.4):

1. **Face validity:** a model has face validity if it appears, to people knowledgeable about the real system, to be a reasonable imitation of it. It is tested by having such users examine model output for reasonableness, which also identifies deficiencies and increases users' confidence in the model.
2. **Validation of model assumptions**, of two kinds. **Structural assumptions** concern how the system operates and is physically arranged and what kind of model should be used (*Example:* how many servers a fast-food drive-through has, and whether they work in parallel or one takes orders/payment while another prepares the order); these should be verified with users of the actual system. **Data assumptions** concern what input data and parameter values the model uses: there must be a sufficient amount of appropriate data, from a reliable source, to build and validate the model, and any outliers must be checked.
3. **Validating input-output transformations:** treat the model as a black box, feed it the same input as the real system, and compare the output of each. If the two agree closely, the black box appears to be working fine; otherwise something is wrong. This requires data recorded while observing the real system.

7.4 Naylor and Finger Validation Process

Naylor and Finger's three-step validation process

1. Build a model that has high face validity.
2. Validate model assumptions.
3. Compare the model's input-output transformations to the corresponding input-output transformations for the real system.

Each step is elaborated in §7.3 above: step 1 is examined by knowledgeable users; step 2 splits into structural and data assumptions; step 3 treats the model as a black box whose output is compared against the real system's output for the same input.

7.5 Validation: Errors

Model validation requires several steps, and each is a potential source of error in the final result. As a general rule, if observed and simulated data disagree, the technical structure of the model should be the *last* factor to suspect.

Six sources of validation error

1. Model inadequate
2. Lack of calibration
3. Errors in the code
4. Errors in the inputs
5. Errors in the use
6. Errors in the experimental data

Possible remedies are given for four of these six sources:

- **Model inadequacy:** check whether all important processes for the given environment are included, whether they are modelled correctly, and whether the range of data used to build the model's components is wide enough to cover the conditions being simulated.
- **Lack of calibration:** continue the calibration process (§7.3) until the model reaches an accepted form.
- **Errors in the code:** cross-check by doing the calculation independently (e.g. in a spreadsheet) and comparing with the model's results; verify that results stay within known physical/biological reality; run the simulation with highly contrasting inputs.
- **Errors in the experimental data:** the experimental data used to test the model's predictive capability are themselves affected by experimental error, which can be large; only a large body of experimental data allows a statistically meaningful evaluation of model performance.

No remedies are given in the source for *errors in the inputs* or *errors in the use*.

Six error sources: model inadequate, lack of calibration, errors in the code, errors in the inputs, errors in the use, errors in the experimental data. This is *not* the Type I/Type II statistical-error framework – that terminology is not used for validation in this course.

Chapter 8 Analysis of Simulation Output

8.1 Confidence Intervals and Hypothesis Testing

Once a stochastic variable is introduced into a simulation, almost every output variable describing the system's behaviour becomes stochastic too, so no single measurement can be taken to represent its value. Most statistical methods also presuppose mutually independent observations, but simulation output is not: a run produces many observed values of a variable, and the value observed at one time is typically influenced by the value observed earlier. This motivates careful statistical treatment of output – estimates should be **consistent** (converge to the true value as sample size grows), bias in both mean and variance estimates should be controlled, and the required run length for a given confidence level must be determined. Because of this randomness, a single point value is not enough on its own; a confidence interval is needed alongside it.

Confidence interval

A **confidence interval (CI)** is a range of values likely to contain the true value of a performance measure at a specified confidence level (e.g. 95%), reported instead of a single point value.

$$\text{For } n \geq 30 : \quad CI = \bar{x} \pm Z \frac{s}{\sqrt{n}} \qquad \text{For } n < 30 : \quad CI = \bar{x} \pm t \frac{s}{\sqrt{n}}$$

where $\bar{x}$ is the sample mean, s the sample standard deviation, n the number of observations (simulation runs), Z the standard-normal critical value (1.96 for 95%), and t the critical value from the t -distribution (used when $n < 30$, with $n - 1$ degrees of freedom).

Example: constructing a 95% confidence interval from simulation output.

Six independent simulation replications of a queueing model give the following average waiting times (minutes) per run:

$$8, 10, 9, 11, 7, 9$$

Step 1 – sample mean.

$$\sum x_i = 8 + 10 + 9 + 11 + 7 + 9 = 54, \quad \bar{x} = \frac{54}{6} = 9 \text{ min}$$

Step 2 – deviations and squared deviations ($x_i - \bar{x}$, then $(x_i - \bar{x})^2$):

$$-1, 1, 0, 2, -2, 0 \quad \longrightarrow \quad 1, 1, 0, 4, 4, 0$$

$$\sum (x_i - \bar{x})^2 = 1 + 1 + 0 + 4 + 4 + 0 = 10$$

Step 3 – sample variance and standard deviation ($n = 6$, so $n - 1 = 5$).

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1} = \frac{10}{5} = 2, \quad s = \sqrt{2} \approx 1.4142$$

Step 4 – standard error of the mean.

$$SE = \frac{s}{\sqrt{n}} = \frac{1.4142}{\sqrt{6}} = \frac{1.4142}{2.4495} \approx 0.5774$$

Step 5 – critical value. Since $n = 6 < 30$, use the t -distribution with $n - 1 = 5$ degrees of freedom at the 95% level: $t_{0.025, 5} \approx 2.571$.

Step 6 – margin of error and the interval.

$$ME = t \times SE = 2.571 \times 0.5774 \approx 1.485$$

$$CI = \bar{x} \pm ME = 9 \pm 1.485 = (7.515, 10.485)$$

We are 95% confident the true mean waiting time lies between about 7.52 and 10.49 minutes.

Hypothesis testing

Hypothesis testing is a statistical method used to decide whether simulation data provide enough evidence to support or reject a claim about a system – i.e. whether an observed difference is real (statistically significant) or simply random variation in the simulation output.

1. **State the hypotheses:** null hypothesis H_0 (assumes no difference or effect) versus alternative hypothesis H_1 (assumes a difference or effect exists).
2. **Choose the significance level α** (usually 0.05 or 0.01) – the maximum acceptable probability of making a Type I error.
3. **Collect simulation data:** run the simulation multiple times and record the performance measure (e.g. waiting time, throughput, utilisation).
4. **Calculate the test statistic:** depending on the problem, a z -test, t -test, chi-square test, or F -test.
5. **Find the p -value or critical value:** compare the test statistic with the critical value, or the p -value with α .
6. **Make a decision:** if $p\text{-value} \leq \alpha$, reject H_0 ; if $p\text{-value} > \alpha$, fail to reject H_0 .

8.2 Estimation Methods

Since infinitely many simulation results cannot be observed, a finite sample is used to **estimate** the true value of a performance measure (e.g. average waiting time or average queue length). Suppose the underlying random variable is drawn from an infinite population with a stationary distribution of finite mean μ and finite variance σ^2 – the population is unaffected by how many samples have already been drawn and does not change over time – and, further, that the value of one sample does not affect the value of any other. Random variables meeting both conditions are **independent and identically distributed (i.i.d.)**, for which the **central limit theorem** applies: the sum of n i.i.d. variables, drawn from a population with mean μ and variance σ^2 , is approximately normally distributed with mean $n\mu$ and variance $n\sigma^2$.

Any normal distribution can be transformed into a standard normal distribution (mean 0, variance 1). For x_i ($i = 1, \dots, n$) i.i.d.:

$$z = \frac{\sum_{i=1}^n x_i - n\mu}{\sqrt{n} \sigma}$$

Dividing numerator and denominator by n gives the sample-mean form used throughout this chapter:

$$z = \frac{\bar{x} - \mu}{\sigma / \sqrt{n}}, \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Point estimation gives a single best-guess value for a performance measure (e.g. $\bar{x}$ estimating μ); **interval estimation** gives a range around it – the confidence interval of §8.1 – that also conveys how precise the estimate is. A good estimator should be consistent (converges to the true value as n grows) and should keep bias in both the mean and the variance it reports under control.

8.3 Simulation Run Statistics

Consider a single-server queue in which arrivals follow a Poisson distribution and service times are exponentially distributed – an **M/M/1** system (Markovian inter-arrival time, Markovian service time, one server), and suppose the objective is to measure the mean waiting time: the time entities spend waiting for service, excluding the service itself.

Within-run statistic. The simplest estimate accumulates the waiting time of n successive entities and divides by n :

$$\bar{x}(n) = \frac{1}{n} \sum_{i=1}^n x_i$$

written $\bar{x}(n)$ to stress that its value depends on how many observations n are taken; each x_i includes the value 0 for entities that do not have to wait.

Autocorrelation. Whenever a queue forms, each entity's waiting time clearly depends on the waiting times of those ahead of it. Data with this property – one value affecting later values – is **autocorrelated**. $\bar{x}(n)$ remains a satisfactory estimate of the mean even though the underlying data are autocorrelated rather than independent, and its sampling distribution approaches normal as the sample size increases.

Initial-state effect. A simulation run is started in some initial state, frequently the **idle** state (no entity being served, none waiting). Early arrivals then have a more-than-normal probability of obtaining service quickly, so a sample mean that includes these early arrivals is biased. This within-run problem (biased early observations, autocorrelated later ones) is exactly what motivates replication (§8.4) to obtain independent observations, and the elimination of initial bias (§8.5) to remove the early distortion.

8.4 Replication of Runs

Replication runs the same simulation model multiple times using different random-number streams. The model and its input parameters stay the same, but different random numbers produce different output, so each run can be treated as an independent experiment. Performing multiple independent runs lets us obtain more reliable results, reduce the effect of randomness, estimate the true mean and variance, and construct confidence intervals – which a single autocorrelated run cannot safely provide by itself.

Suppose the experiment is repeated p times with independent random-number streams, and let x_{ij} be the i -th observation of the j -th run ($i = 1, \dots, n; j = 1, \dots, p$). The sample mean and variance for the j -th run are:

$$\bar{x}_j(n) = \frac{1}{n} \sum_{i=1}^n x_{ij}, \quad s_j^2 = \frac{1}{n-1} \sum_{i=1}^n [x_{ij} - \bar{x}_j(n)]^2$$

Combining the results of the p independent replications gives overall estimates of the population mean and variance:

$$\bar{x} = \frac{1}{p} \sum_{j=1}^p \bar{x}_j(n), \quad s^2 = \frac{1}{p} \sum_{j=1}^p s_j^2(n)$$

$\bar{x}$ estimates the mean waiting time, and s^2 is used to establish a confidence interval with $p - 1$ degrees of freedom.

Run length n and replication count p trade off against each other for a fixed total sample size $N = pn$. More, shorter replications narrow the confidence interval but increase the relative effect of each run's starting conditions (§8.5); fewer, longer replications reduce that effect but leave a wider interval, and results are inaccurate if the runs are not properly initialised. There is no established rule for splitting N into p and n – only that p should not be too large, and the replication means $\bar{x}_j(n)$ should approximate a normal distribution (central limit theorem, §8.2) for the interval formulas to be valid.

8.5 Elimination of Initial Bias

Initial bias

Initial bias is a condition that occurs when the initial values of the state variables in a simulation do not represent the system's long-run (steady-state) behaviour, so that the output collected at the start of a run is unrealistic and biases the estimated performance measures. The goal is **intelligent initialisation**: selecting initial conditions that are more representative of the steady-state distribution.

Two general approaches remove the bias: start the system in a more representative state than empty, or ignore the first part of the simulation.

Representative initialisation. If the real system exists, data collected on it can be used to specify more typical initial conditions; this requires a large data-collection effort and is impossible if the system being modelled does not yet exist, though for complex systems, data from a second, simplified model can be used instead. Ideally the steady-state distribution is known, and the initial condition is drawn directly from it.

Deletion of initial data (warm-up period). The more common approach is to eliminate an initial section of the run. Each run is divided into an **initialisation phase**, from time 0 to T_0 , and a **data-collection phase**, from T_0 to the stopping time $T_0 + T_E$. The run is started idle and stopped at T_0 ; it is then restarted with statistics gathered only from that point on – in practice the simulation is programmed to gather statistics from time 0 throughout, and the statistics gathered before T_0 are simply discarded. No simple rule fixes how long an interval should be eliminated. The cost of this method: the variance estimate needed for a confidence interval is based on less information once the initial data are discarded, so the reduction in bias is bought at the price of a wider confidence interval.

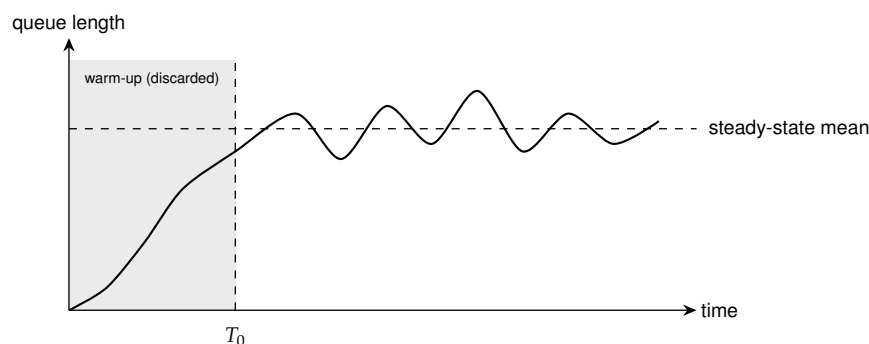


Figure 22: Initial transient: the shaded warm-up period before T_0 is discarded; data from T_0 onward fluctuate around the steady-state mean and are kept for output statistics.

Four ways to handle initial bias, as given for this system: (1) ignore the initial bias that occurs during the run; (2) start the system in a representative state rather than the empty state; (3) start in the empty state, stop after the initial bias has passed, then restart statistics collection; (4) run the simulation for such a long period that the initial bias has no significant effect on the output result.

Chapter 9 Simulation Software

Software for building simulation models falls into three broad categories: **general-purpose programming languages** (C, C++, Java), **simulation programming languages** or SPLs (GPSS/H, SIMAN V, SLAM II), and **simulation environments** (GUI-based packages such as Arena or Simul8). GPSS, the first process-interaction SPL, was built at IBM in 1961 for quick modelling of communication and computer systems; later decades added activity-scanning/event-scheduling support (GASP), multiple-worldview languages (SLAM II, SIMAN), and finally integrated GUI environments with animation and statistical analyzers.

Choosing a package. Do not judge on ease of use alone – weigh accuracy/level of detail, ease of learning, vendor support and fit to the application; execution speed matters; be wary of glossy demos and yes/no “checklists” (a vendor’s “conveyor entity” may be far weaker than another’s); ask the vendor to solve a small version of *your* problem; and check whether the package is powerful enough on its own, or whether “no programming required” really holds for your case.

Throughout this section the same running problem is used to compare languages: a **grocery checkout counter** modelled as a single-server queue, run until **1000 customers** have been served, with **exponential** inter-arrival time (mean 4.5 min) and **normal** service time (mean 3.2 min, s.d. 0.6 min); when the cashier is busy a queue forms and no customer is turned away.

9.1 Simulation in Java

Java is a general-purpose language with no built-in simulation constructs, but it is widely used for simulation because it ships a random-number generator, supports modular design, and can be extended with simulation libraries (e.g. **SSG**) that hide low-level event-scheduling bookkeeping.

Components of a Java simulation model

Basic: system state, entities/attributes, sets, events, activities, delays. **Common (modular) components:** a **Clock** variable; an **Initialization** method (state at time 0); a **min-time-event** method that scans the future event list (FEL) for the smallest time stamp; one **event method** per event type; **random-variate generators**; a **main program** driving the event-scheduling loop; and a **report generator**.

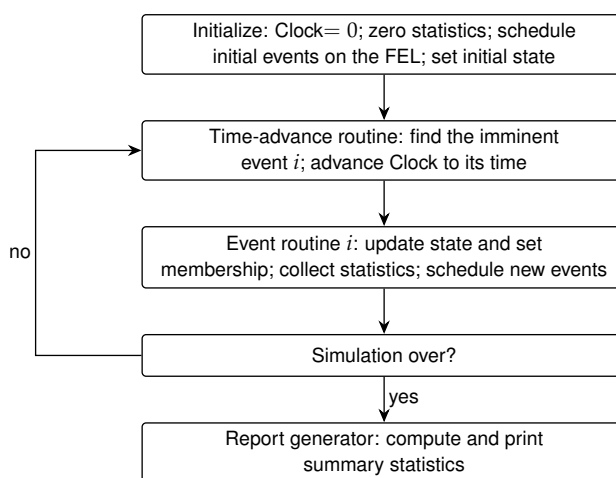


Figure 23: Generic event-scheduling algorithm underlying a Java (or any general-purpose-language) simulation.

Example: For the checkout counter, the state variables are `QueueLength` and `NumberInService`; the entity set is `Customers` (an FCFS queue); the FEL holds pending arrival/departure events;

inputs are MeanInterArrivalTime= 4.5, MeanServiceTime= 3.2, SIGMA= 0.6, TotalCustomers= 1000; accumulators include TotalBusy, MaxQueueLength, SumResponseTime, NumberOfDepartures and LongService (customers who spent ≥ 4 min in the system); and Exponential(μ), Normal($x\mu$, SIGMA) generate the two random variates. The event methods are ProcessArrival, ProcessDeparture, driven by Initialization and finished by ReportGeneration. A skeleton implementing this exact structure:

```
double clock = 0.0, meanInterArrival = 4.5, meanService = 3.2, sigma = 0.6;
int totalCustomers = 1000, numberOfDepartures = 0, queueLength = 0;

PriorityQueue<Event> futureEventList = new PriorityQueue<>();
futureEventList.add(new Event("ARRIVAL", exponential(meanInterArrival)));

while (numberOfDepartures < totalCustomers) {
    Event imminent = futureEventList.poll(); // min-time event
    clock = imminent.time;                  // advance Clock
    if (imminent.type.equals("ARRIVAL")) processArrival();
    else processDeparture();
}
reportGeneration();

void processArrival() {
    queueLength++;
    futureEventList.add(new Event("ARRIVAL", clock + exponential(meanInterArrival)));
    if (queueLength == 1) // server was idle
        futureEventList.add(new Event("DEPARTURE", clock + normal(meanService, sigma)));
}

void processDeparture() {
    queueLength--;
    numberOfDepartures++;
    if (queueLength > 0)
        futureEventList.add(new Event("DEPARTURE", clock + normal(meanService, sigma)));
}
```

exponential(μ) and normal(μ , sd) stand in for the slide's Exponential(μ)/Normal($x\mu$, SIGMA) generators; the point is the event-scheduling *structure* (Clock, FEL, event routines), not a complete runnable program.

9.2 Simulation in GPSS

GPSS

GPSS (General Purpose Simulation System) is a highly structured, special-purpose simulation language built on the **process-interaction** worldview and oriented toward queueing systems. A model is drawn as a **block diagram** (over 40 standard block types, each representing an event, delay or other action on transaction flow); the diagram is then written as block statements, control statements are added, and the result is a GPSS model. Released by IBM in 1961; **GPSS/H** (1977) is the most widely used version today.

Control statements (a sample): START sets the termination count and begins the run; RESET/CLEAR reset statistics; REPORT names or requests the report file. **Block reference** (compact):

Block	Meaning	Block	Meaning
GENERATE	creates transactions entering the model	GATE	routes a transaction on an entity's state
ADVANCE	delays a transaction a given time	LOGIC	sets/resets a logic switch
ASSIGN	sets a transaction parameter	MARK	timestamps a transaction parameter
QUEUE	increments a queue's statistics	PRIORITY	sets transaction priority
DEPART	decrements a queue's statistics	TABULATE	records a value in a table entity
SEIZE	takes ownership of a facility	ENTER	takes storage units from a store
RELEASE	gives up a facility	LEAVE	returns storage units to a store
TERMINATE	removes the transaction; decrements the term. count	TEST	branches on comparing two values
TRANSFER	sends the transaction to another block		

Example: Grocery checkout counter as a GPSS model – the same scenario as §9.1, translated directly into the standard single-server-queue GPSS pattern:

GENERATE	4.5,0	; customers arrive, exponential mean 4.5 min
QUEUE	LINE	; join the checkout queue
SEIZE	CLERK	; wait for / acquire the cashier (facility)
DEPART	LINE	; leave the queue, update queue statistics
ADVANCE	3.2,0.6	; service time, mean 3.2 min, spread 0.6
RELEASE	CLERK	; free the cashier
TERMINATE	1	; customer leaves; decrement termination count
START	1000	; run until 1000 customers have terminated

Line	What it does
GENERATE 4.5,0	Injects a new customer transaction every exponential(4.5) min.
QUEUE LINE	Customer joins queue entity LINE; queue-length statistics updated.
SEIZE CLERK	Customer waits for, then takes ownership of, facility CLERK; if busy, the transaction rests on the facility's delay chain.
DEPART LINE	Customer leaves LINE; queue content and wait-time statistics updated.
ADVANCE 3.2,0.6	Models the service time at the counter.
RELEASE CLERK	Cashier freed for the next waiting customer.
TERMINATE 1	Customer exits the model; termination counter decremented by 1.

The START 1000 control statement matches the “simulate 1000 customers” stopping rule.

9.3 Simulation in Python

Python is a general-purpose language with no special compiler or simulation syntax, but a rich ecosystem – **SimPy** (process-based discrete-event simulation), **SciPy/NumPy** (numerical modelling) and **Matplotlib** (visualisation) – gives it the flexibility of a general-purpose language together with dedicated simulation libraries.

SimPy

SimPy is an open-source, process-based DES library built around five concepts: (1) **Environment** – manages simulation time and event scheduling; (2) **Process** – an active entity or sequence of actions; (3) **Event** – something that triggers after a delay or condition; (4) **Resource** – a shared, limited resource such as a server; (5) **Timeout** – delays a process for a given simulated duration.

Why Python for simulation: easy, readable syntax; a rich library ecosystem (SimPy, NumPy, SciPy, Pandas, Matplotlib); supports discrete-event, continuous, Monte Carlo and agent-based models; strong data analysis/visualisation of results; free and cross-platform.

Example: The grocery checkout counter in SimPy – a checkout_counter **Resource** of capacity 1, and a customer **Process** that requests it after an exponential inter-arrival **Timeout** and is served for a normally distributed **Timeout**:

```

import simpy, random

MEAN_IAT, MEAN_SVC, SD_SVC = 4.5, 3.2, 0.6    # minutes

def customer(env, cid, checkout_counter):
    with checkout_counter.request() as req:    # Event: wait for the Resource
        yield req
        service_time = random.normalvariate(MEAN_SVC, SD_SVC)
        yield env.timeout(service_time)        # Timeout: being served

def customer_arrivals(env, checkout_counter):
    for cid in range(1000):
        yield env.timeout(random.expovariate(1 / MEAN_IAT)) # Timeout: inter-arrival
        env.process(customer(env, cid, checkout_counter)) # Process: new customer

env = simpy.Environment()                    # Environment: clock + scheduler
checkout_counter = simpy.Resource(env, capacity=1) # Resource: the single cashier
env.process(customer_arrivals(env, checkout_counter))
env.run()

```

Note the same parameters as the Java version (mean inter-arrival 4.5, mean service 3.2, s.d. 0.6, 1000 customers) so the two implementations model exactly the same system.

9.4 Other Simulation Software

Beyond general-purpose languages and SPLs, many commercial **simulation environments** run on a PC with a GUI, animation and automatically collected output statistics (often including confidence intervals). Most take the process-interaction worldview, a few also support event-scheduling or mixed discrete-continuous models, and animation ranges from scale 2-D/3-D drawings to iconic process-flow diagrams.

Tool	Brief description
Arena	Built on the SIMAN language; Basic/Standard/Professional editions; Input Analyzer fits distributions, Output/Process Analyzer compares design alternatives; OptQuest add-on for optimisation.
AutoMod	Suite = AutoMod (modelling) + AutoStat (experimentation) + AutoView (3-D animation); manufacturing and material-handling focus, with reusable “systems.”
Delmia/QUEST	Manufacturing-oriented 3-D simulation with CAD geometry; includes IGRIP (robot simulation) and PROCESS ENGINEER; scriptable via SCL/BCL.
Extend	Block-diagram model building plus a compiled block-authoring language; discrete, continuous and mixed models; open architecture, can call external code.
Flexsim	Discrete-event, object-oriented (C++/OpenGL); 2-D/3-D/VR animation; used for capacity, bottleneck and line-balancing studies.
Micro Saint	General-purpose discrete-event network simulator with industry-neutral terminology; Network diagram + Actionview views; bundled OptQuest optimisation.
ProModel	Manufacturing-oriented; entities/locations/resources/paths/logic/arrivals; 2-D (quasi-3-D) animation; related MedModel (healthcare), ServiceModel (services).
Simul8	Drag-and-drop icons/arrows for flow, queues and resources; aimed at service-industry transaction processing.
WITNESS	Manufacturing and service editions; models built from reusable template/module elements; multi-window 2-D layout animation.

Experimentation and statistics add-ons. Almost all packages offer output statistics, and many add **optimisation**: the user defines an objective (e.g. cost) and the tool searches for a near-optimal design. Examples: Arena’s **OptQuest** (scatter search + tabu search + neural-network screening to

escape local optima); AutoMod's **AutoStat** (evolutionary strategies, warm-up detection, design of experiments, common random numbers, distributed runs); ProModel's **SimRunner** (genetic algorithms/evolutionary strategies, also estimates warm-up length and needed replications).

Chapter 10 Simulation of Computer Systems

Computer systems are enormously complex, with behaviour spanning time scales from a transistor switching ($\sim 10^{-11}$ s) to a human interacting with the machine (seconds to minutes). To manage this, computer systems are designed – and hence simulated – **hierarchically**: each level zooms in on one component of the level above it.

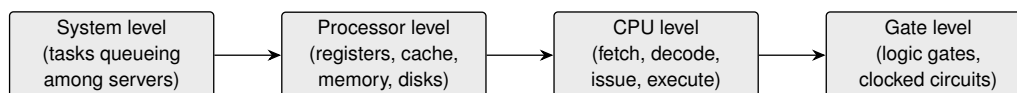


Figure 24: Levels of abstraction in computer-system simulation; each level zooms in on one component of the previous level. Sections 10.2–10.4 work through the system, CPU and memory levels.

10.1 Simulation Tools

Different levels of abstraction call for different tools, and a tool’s key strength is how it helps in **model building**. At a low level, **VHDL** describes digital electronic systems and serves as both design specification (topology) and simulation specification (behaviour), with support for modular design.

Simulation languages are less common in academic computer science than in industry, partly on cost and partly on emphasis: a dedicated simulation language ships a rich set of predefined simulation objects/actions with a programming language available underneath to describe their behaviour, whereas a general-purpose language with simulation *constructs* defines only a few elemental simulation objects and expresses most of the model through ordinary control flow. Some tools are so richly preprogrammed that a model needs no code at all; others require substantial user programming. Either way, a fundamental characteristic of any tool that supports user-programmed behaviour is its **worldview** – process-interaction, event-scheduling or activity-scanning (as introduced earlier in the course, e.g. GPSS is process-interaction).

10.2 High-Level Computer-System Simulation

At the system level, a computer system can be viewed as a **queuing network**: jobs or requests are customers, and CPUs/servers/disks are service centres with queues – exactly the servers-and-queues picture of Chapter 4, though here the “customers” are website requests rather than people. The two questions this level of simulation must answer are: what is the **response time**, and where is the **bottleneck**?

Example: Web-site server system. A company wants a capacity-planning study of the system behind its website. A load-balancing **router** distributes incoming requests to **web servers**, which manage interaction with the outside world; web servers pass work over a local area network to **application servers** running the site’s applications; application servers in turn request services from **data servers**, which handle queries and updates against a **data-disk** tier.

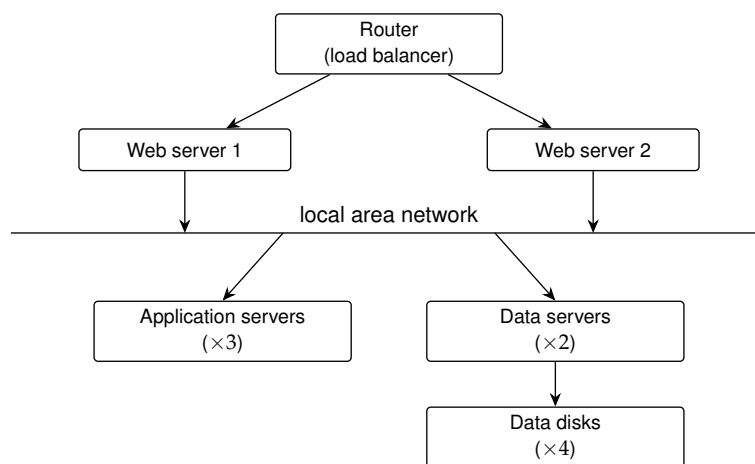


Figure 25: Web-site server system as a queuing network: router → web servers → LAN → application servers / data servers → disks.

The router keeps a table of sessions; each web server has three queues of threads and each application server two. The study first finds the **response-time distribution** and the bottleneck, then explores relief through scheduling-policy changes, biasing applications toward particular servers, or adding CPU/IO capacity.

Subsystem	What must be specified
Router	Load-balancing policy, execution times
Web server	Server count, queuing policy, execution times
Application server	Server count, queuing policy
Data server	Server count, disk count, queuing policy, disk time

10.3 CPU Simulation

One level down, execution time is no longer an input constant (as it was in §10.2) – it is what the simulation exists to **discover**, given a stream of instructions and the CPU’s logical design. The central design problem is avoiding **stalls**: a CPU stalls whenever it commits to an instruction whose inputs are not yet ready, and the leading cause is memory latency (which can run to tens of CPU cycles).

Example: Consider load R2, 4(R3) (read memory at address R3+4 into R2), followed later by add R4, R2, R5 (compute R4 = R2 + R5). If the CPU insisted on completing the load before doing anything else it could stall for a long time on a cache miss; a high-performance CPU instead keeps executing other instructions and only stalls when it reaches an instruction – here the add – that actually needs R2. This is the idea behind **instruction-level parallelism (ILP)** and **pipelining**: each instruction passes sequentially through several stages, with dedicated hardware per stage, so several instructions are in flight at once.

Pipeline stages

Fetch (read the instruction from memory/cache) → **Decode** (identify the operation and registers involved) → **Issue** (release the instruction once nothing holds it back) → **Execute** (perform the operation) → **Complete** (write the result to the destination register) → **Graduate** (retire instructions in original program order).

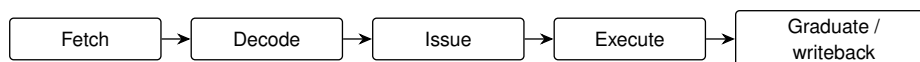


Figure 26: The five pipeline stages a simulated instruction moves through.

Example: **Pipeline timing.** With no stalls, a new instruction can be fetched every cycle

while earlier instructions occupy the later stages – each row below is one instruction, each column one clock cycle, and the letter is the stage occupied that cycle (F/D/I/E/G):

	1	2	3	4	5	6	7	8
I1	F	D	I	E	G			
I2		F	D	I	E	G		
I3			F	D	I	E	G	
I4				F	D	I	E	G

Instruction I1 graduates in cycle 5, and thereafter one instruction graduates per cycle – the throughput a pipeline simulation is built to verify (and to show where a stall, e.g. a memory-latency wait at Fetch or Issue, would break this diagonal pattern).

What a pipeline simulation must model at each stage:

- **Fetch** – read from simulated memory and cache.
- **Decode** – register mapping; branch prediction.
- **Issue** – checks that input registers *and* functional units are available.
- **Execute & complete** – resolve branches; write registers; release functional units/registers.
- **Graduate** – check for exceptions, in program order.

10.4 Memory Simulation

The growing speed gap between CPU and main memory is addressed with a **memory hierarchy**: a small L1 cache runs at CPU speed, a larger and slower L2 cache backs it, and main memory (larger, slower still) sits behind that. Data recently referenced – and nearby data likely to be referenced soon – is kept in the smaller, faster levels; it moves up on demand and ages out as it falls out of use.

Improving hierarchy quality

The main quality measure of a memory hierarchy is its **hit ratio** at each level. **Set-associative** caches use associative memory to map search keys to data; a **fully-associative** cache lets any address occupy any cache slot. When a referenced block is not found (a **miss**) and the set is full, a **replacement policy** picks a victim to evict – under **LRU** (Least Recently Used), the block evicted is the one referenced longest ago among those in the set.

Example: LRU page replacement, worked. Reference string 7, 0, 1, 2, 0, 3, 0, 4 with 3 page frames. At each step the least-recently-used resident page is evicted if a fault occurs and the frames are full:

Reference	7	0	1	2	0	3	0	4
Frame 1	7	7	7	0	0	0	0	0
Frame 2		0	0	1	1	2	2	3
Frame 3			1	2	2	3	3	4
Fault?	F	F	F	F	–	F	–	F

Step 4 (page 2) evicts 7, the least recently used of {7, 0, 1}; step 5 (page 0) is a **hit** so nothing is evicted; step 6 (page 3) evicts 1, now the least recently used of {0, 1, 2}; step 7 (page 0) hits again; step 8 (page 4) evicts 2, the least recently used of {0, 2, 3}. Total: **6 page faults** out of 8 references (2 hits, at references 5 and 7).

10.5 Simulation of Computer Networks

Network simulation

Computer network simulation builds a virtual model of a network to study its behaviour – performance, communication delay, packet loss, throughput, congestion, reliability – without constructing the real network.

Reasons to simulate: predict performance before deployment, compare protocols, identify bottlenecks, cut implementation cost, and test large-scale networks safely.

Components of a network simulation model: **nodes** (computers, routers, switches, servers), **links** (wired or wireless channels), **packets** (units of data), **traffic** (data generated by users/applications), a **routing protocol** (determines packet paths), and **queues/buffers** that hold packets awaiting transmission. This last component is the network-simulation echo of the same queuing ideas used throughout the course: packets queue at router buffers exactly as customers queue at a server, and “identify bottlenecks”/“communication delays” are just response-time and utilisation questions asked at the packet level.

Tool	Notes
NS-3	Discrete-event, packet-level network simulator widely used in research.
OMNeT++	Component-based, modular discrete-event network simulation framework.
Cisco Packet Tracer	Educational network-design and behaviour simulator.
GNS3	Emulates real router/switch software for network-engineer training.
Mininet	Creates lightweight virtual networks of hosts/switches for experimentation.
Boson NetSim	Router/switch simulator used for networking exam preparation.

Summary. System-level simulation treats a computer system as a queuing network of servers (§10.2); CPU simulation drops a level to ask how an instruction stream actually flows through a pipelined design, cycle by cycle (§10.3); memory simulation asks how well a cache hierarchy exploits locality under a replacement policy such as LRU (§10.4); and network simulation applies the same discrete-event, queue-and-server thinking at the packet level (§10.5).

Knowledge Check: 40 Multiple Choice Questions

Choose one option per question. Answers are at the end. Questions 1–3 cover Chapter 1, 4–6 Chapter 2, 7–11 Chapter 3, 12–16 Chapter 4, 17–19 Chapter 5, 20–28 Chapter 6, 29–31 Chapter 7, 32–34 Chapter 8, 35–37 Chapter 9 and 38–40 Chapter 10.

Chapter 1: Introduction to Simulation

1. A model in which the state variables change continuously with time is called a:

(A) discrete model	(B) static model
(C) continuous model	(D) stochastic-free model
2. The first step in a formal simulation study is normally:

(A) model verification	(B) model validation
(C) experimentation with the model	(D) problem formulation and setting of objectives
3. Monte Carlo simulation primarily relies on:

(A) solving the governing differential equations analytically	(B) purely deterministic step-by-step logic
(C) analog computer block diagrams only	(D) repeated random sampling to obtain numerical results

Chapter 2: Physical and Mathematical Models

4. For the system $M\ddot{x} + D\dot{x} + Kx = KF(t)$, the system is critically damped when:

(A) $D^2 = 4MK$	(B) $D^2 > 4MK$
(C) $D^2 < 4MK$	(D) $D = 0$
5. In the mechanical-electrical analogy, the electrical quantity analogous to mechanical damping D is:

(A) inductance L	(B) capacitance C
(C) resistance R	(D) charge q
6. A static mathematical model describes a system:

(A) as it evolves continuously over time	(B) at a specific point in time, independent of time-varying behaviour
(C) only through a physical scale prototype	(D) only when the system is unstable

Chapter 3: Simulation of Continuous Systems

7. In an analog computer, integration of a signal with respect to time is performed by:

(A) a summer block	(B) a sign-changer block
(C) an integrator block	(D) a scale-factor (potentiometer) block
8. With $M = 1$, $D = 6$, $K = 9$, the system $M\ddot{x} + D\dot{x} + Kx = KF(t)$ is:

(A) overdamped	(B) underdamped
(C) critically damped	(D) unstable
9. Hybrid simulation combines:

(A) analog and digital computation	(B) two independent digital computers only
(C) GPSS and Java code	(D) Monte Carlo sampling and Markov chains
10. CSSL stands for:

(A) Computer System Simulation Logic	(B) Continuous State-Space Loop
(C) Continuous System Simulation Language	(D) Calibrated Simulation Scripting Language

11. In a negative-feedback control loop, the fed-back signal is subtracted from the reference input to form:
- (A) the output signal directly
 - (B) the error signal
 - (C) the disturbance input
 - (D) the controller gain

Chapter 4: Simulation of Queuing Systems

12. In Kendall's notation $A/B/c/N/K/D$, the symbol c represents:
- (A) the arrival distribution
 - (B) the system capacity
 - (C) the queue discipline
 - (D) the number of servers
13. For an $M/M/1$ queue with $\lambda = 8$ and $\mu = 10$, the traffic intensity ρ is:
- (A) 1.25
 - (B) 0.8
 - (C) 0.08
 - (D) 8.0
14. For an $M/M/1$ queue, the expected number of customers in the system is:
- (A) $L = \frac{\rho}{1 - \rho}$
 - (B) $L = \frac{\rho^2}{1 - \rho}$
 - (C) $L = \lambda\mu$
 - (D) $L = 1 - \rho$
15. Little's Law relates L , λ and W as:
- (A) $L = W/\lambda$
 - (B) $L = \lambda W$
 - (C) $W = \lambda/L$
 - (D) $L = \lambda + W$
16. A queuing system in which each server has its own dedicated queue is called a:
- (A) multiple-queue, multiple-server system
 - (B) single-queue, multiple-server system
 - (C) tandem (series) queue network
 - (D) closed queuing network

Chapter 5: Markov Chains

17. The Markov (memoryless) property states that the next state depends on:
- (A) the entire history of past states
 - (B) only the initial state at $n = 0$
 - (C) the determinant of the transition matrix
 - (D) only the current state, not the earlier history
18. For the n -step transition probabilities, the matrix $P^{(n)}$ equals:
- (A) P^n , the n -th power of the one-step matrix
 - (B) nP
 - (C) P/n
 - (D) $P + n$
19. The steady-state distribution π of a Markov chain satisfies:
- (A) $P\pi = \mathbf{0}$
 - (B) $\pi = P^{-1}$
 - (C) $\pi P = \pi$ with $\sum_i \pi_i = 1$
 - (D) $\sum_i \pi_i = 0$

Chapter 6: Random Number Generation

20. A good random-number generator must chiefly satisfy the two properties of:
- (A) high mean and low variance
 - (B) uniformity and independence
 - (C) determinism and short period
 - (D) symmetry and skewness
21. The linear congruential generator recurrence is:
- (A) $X_{i+1} = aX_i/m$
 - (B) $X_{i+1} = (aX_i) \bmod c$
 - (C) $X_{i+1} = (X_i + c)/a$
 - (D) $X_{i+1} = (aX_i + c) \bmod m$
22. A multiplicative congruential generator is the special case of the mixed congruential generator where:
- (A) $a = 0$
 - (B) $c = 0$
 - (C) $m = 0$
 - (D) $X_0 = 0$
23. Using $X_{i+1} = (5X_i + 1) \bmod 8$ with $X_0 = 1$, the value of X_1 is:
- (A) 5
 - (B) 1
 - (C) 6
 - (D) 7

24. The Kolmogorov–Smirnov test compares:

- (A) the maximum absolute difference between the empirical and theoretical CDFs
 (B) only the means of two samples
 (C) the variance ratio of two samples
 (D) the mode of the sampled data

25. The chi-square goodness-of-fit statistic is computed as:

- (A) $\sum (O_i - E_i) / E_i$
 (B) $\sum (O_i - E_i)^2 / E_i$
 (C) $\sum O_i / E_i$
 (D) $\sum (O_i - E_i)^2$

26. In the poker test on 3-digit numbers, the three possible digit patterns are:

- (A) uniform, skewed, bimodal
 (B) low, medium, high
 (C) all different, exactly one pair, all three alike
 (D) none, one, or two pairs only

27. The inverse-transform method generates a variate X by:

- (A) setting $X = F(R)$ directly
 (B) rejecting values that fall above a bounding curve
 (C) summing several independent uniform variates
 (D) setting $X = F^{-1}(R)$ for R uniform on $[0, 1)$

28. For an exponential distribution with rate λ , the inverse-transform formula gives:

- (A) $X = \ln(R) / \lambda$
 (B) $X = -\ln(1 - R) / \lambda$
 (C) $X = 1 - e^{-\lambda R}$
 (D) $X = \lambda \ln(R)$

Chapter 7: Verification and Validation

29. Verification of a simulation model asks whether:

- (A) the model represents the right real system
 (B) the model has been built right (it is internally correct)
 (C) the model runs faster than real time
 (D) enough replications have been made

30. Validation of a simulation model asks whether:

- (A) the source code compiles without errors
 (B) the random-number generator has a long period
 (C) the model has been verified
 (D) the right model has been built (it represents the real system accurately)

31. The first step of the Naylor and Finger validation process is to:

- (A) build a model that has high face validity
 (B) compare only the model's input-output transformations
 (C) validate the model's assumptions first
 (D) skip expert review of the model

Chapter 8: Analysis of Simulation Output

32. A confidence interval for a simulation output is typically built from:

- (A) only the sample mean
 (B) only the population variance
 (C) the sample mean, sample standard deviation and a critical t (or z) value
 (D) the mode of the replications

33. The initial transient (warm-up) period in a simulation run is usually handled by:

- (A) ignoring the issue entirely
 (B) running only a single replication
 (C) deleting the initial biased observations before collecting statistics
 (D) doubling the arrival rate

34. Independent replications of a simulation run are used mainly to:

- (A) reduce the mean service time
 (B) remove the need for random numbers
 (C) increase the arrival rate
 (D) obtain independent samples for constructing confidence intervals

Chapter 9: Simulation Software

35. In GPSS, the block that creates transactions entering the model is:
 (A) GENERATE (B) TERMINATE
 (C) QUEUE (D) SEIZE
36. In GPSS, after SEIZE takes a facility, the block that models the service (holding) time is:
 (A) ADVANCE (B) DEPART
 (C) RELEASE (D) GATE
37. SimPy models simulation entities mainly using:
 (A) GPSS blocks translated line by line (B) Java threads exclusively
 (C) analog-computer building blocks (D) Python generator-based processes and resources

Chapter 10: Simulation of Computer Systems

38. In a 5-stage instruction pipeline (Fetch, Decode, Issue, Execute, Graduate) with no stalls, in steady state a new instruction can be fetched:
 (A) every 5 clock cycles (B) every clock cycle
 (C) only after the previous instruction graduates (D) only once per program
39. Under the LRU page-replacement policy, when a fault occurs and all frames are full, the evicted page is the one that:
 (A) was most recently used (B) will be needed soonest in the future
 (C) has the smallest page number (D) was referenced longest ago (least recently used)
40. Discrete-event, packet-level simulation of computer networks is commonly performed with tools such as:
 (A) NS-3 or OMNeT++ (B) GPSS and CSMP only
 (C) SimPy exclusively (D) spreadsheet models only

Answer Key

1	C	2	D	3	D	4	A	5	C
6	B	7	C	8	C	9	A	10	C
11	B	12	D	13	B	14	A	15	B
16	A	17	D	18	A	19	C	20	B
21	D	22	B	23	C	24	A	25	B
26	C	27	D	28	B	29	B	30	D
31	A	32	C	33	C	34	D	35	A
36	A	37	D	38	B	39	D	40	A